Strange Tech: Homomorphic Encryption

Asymmetric bets for curious engineers — Last updated November 8, 2026

Marc Magrans de Abril

DRAFT

DRAFT

Table of contents

Preface **1**

Prologue **3**

Acknowledgments **5**

Disclaimer **7**

1 Introduction **9**
1.1 Why this technology? 9
1.2 A brief history 10

2 Hands on **13**
2.1 Recipe 1: Encrypted mean 14
2.2 Recipe 2: Encrypted variance 16
2.3 Recipe 3: Encrypted linear regression 18
2.4 Recipe 4: The noise wall 19
2.5 Recipe 5: Breaking through the wall 21
2.6 Recipe 6: Encrypted search 24

3 Applications **27**

4 Current limitations **29**

5 Foundations **33**
5.1 Lattices 34
5.2 From lattices to polynomial rings: LWE and RLWE 35
5.3 Packing and SIMD 41
5.4 Encryption and homomorphic operations 47
5.5 Rotations 53

DRAFT

5.6 Noise and levels 60
5.7 Polynomial division and rescaling 64
5.8 Bootstrapping 69
5.9 Other encryption schemes 75

Recommended Readings **79**

References **81**

Solutions **83**

DRAFT

Preface

This book is part of the **Strange Tech** series: standalone books about advanced technologies that most engineers don't know about, but that they might need in the future and that represent a professional opportunity.

A Strange Tech is characterized, first, by having survived the passage of time, passing the Lindy filter discovered by B. Mandelbrot and later popularized by N. Taleb. Bicycles, windows, good wine, and good friends all share this quality. The longer they've been around, the more confident we can be that they'll keep surviving — their conditional life expectancy is proportional to their current age. Likewise, these technologies have been developing and in use in certain fields for a long time, and are therefore likely to remain relevant in the future.

Second, and as a consequence of having passed the Lindy filter, a Strange Tech has robust mathematical foundations that establish it as a bet on the future, and therefore there is a continuous or growing stream of scientific papers, news, and software expanding those foundations and their applications.

Third and last, a Strange Tech is a bet on the future in the sense that the market hasn't fully exploited it yet, and therefore the core knowledge can be learned with relatively little effort and may still contain unexplored niches.

DRAFT

DRAFT

Prologue

Elena, head of innovation at a health insurance company, walks into the room carrying a folder full of post-its. Impeccable as always, buoyed by the confidence of a stellar career, today she looks a bit flustered. She's spent several weeks unsuccessfully trying to close a deal with a cloud analytics provider. Daniel, lead engineer on the data team, is waiting, laptop open, after grabbing a coffee and scrolling through the latest Hacker News headlines. At least his day started well, but a storm seems to be brewing. In this chaotic world, some things are predictable.

Elena: *Daniel, I need the vendor to analyze our patient data for the risk models. But Legal has blocked us. They say medical records are confidential and we can't send them to a third party. Isn't there a way for them to run the analysis without seeing the data?*

Daniel: *Run computations on data you can't see… that sounds like asking the chef to cook without touching the ingredients.*

Elena: *Don't give me metaphors. I need a real solution. The contract gets signed in three weeks or we lose the vendor.*

Daniel: *Hold on, let me think… "cook without seeing the ingredients"… This reminds me of something. A couple weeks ago, over coffee, a colleague told me about a new technology — a wild idea that sounds almost like science fiction. There's something called homomorphic encryption that lets you send encrypted data to the cloud, and the analysis runs on the encrypted data — without decrypting it. You'd think that couldn't possibly work, but it turns out you can perform computations on encrypted information without ever decrypting it. The result is also encrypted, and only we can decrypt it. They never see a single name or a single diagnosis. It's like being able to cook without seeing the ingredients.*

Elena: *And that works? Why doesn't everyone use it?*

DRAFT

Daniel: *Because until recently it was impractical — too slow. But the technology has matured enough for certain use cases. Ours might be one. Want me to look into it?*

Elena: *Yes, of course. Can you have a proposal for me by tomorrow?*

Daniel stares at Elena in disbelief. "By tomorrow." Two words that in project management mean "I haven't understood a single thing you just said, but it sounds like it should be easy." He should have told her we're not on Star Trek, and he's clearly not Scotty. At what point did the phrase "new technology" fail to register?

DRAFT

Acknowledgments

This book didn't come out of nowhere. It exists thanks to the accumulation of past knowledge and wealth, and to the unconditional help of family, friends, and colleagues. Regarding the past, I've done a poor job of acknowledging all the people and institutions who made that accumulation possible and from which this book has benefited, but for the sake of brevity I decided to reference only the documents essential to understanding and going deeper into this field. Regarding people, I can't even imagine how I could have written a single word without the upbringing my parents, Ildefons and Pili, gave me, the love of Bego, my siblings, and my children, or without the laughter and advice of my friends. Any merit belongs to this whole flood of people, both known and anonymous, who contributed. Thank you all for your help, love, and friendship.

DRAFT

DRAFT

Disclaimer

The views and opinions expressed in this book are solely my own and do not necessarily reflect those of my past or present employers, colleagues, or friends. I have made every effort to educate myself, understand the different points of view presented, and study the relevant literature — drawing on AI assistants as a research, drafting, and code-review tool along the way. Even so, I have personally verified every claim, figure, and code snippet before including it. Any errors, inaccuracies, or misinterpretations that remain are entirely my own responsibility, and I sincerely apologize in advance for them.

Above all, the single most important thing is this: don't use anything in this book to launch a spaceship to Mars, put the finishing touches on the quantum teleporter you're building in your garage, or convince your teenage kid to turn off his phone. It won't work.

DRAFT

DRAFT

Chapter 1

Introduction

Homomorphic encryption allows computations to be performed on encrypted data without ever decrypting it. This opens the door to a paradigm where privacy and cloud computing are no longer opposing concepts. The price: homomorphic operations are still between 10^4 and 10^6 times slower than their plaintext equivalents, although specialized hardware and dedicated compilers can narrow that gap to 10–100× (Chapter 4), limiting use to scenarios where privacy justifies the cost.

1.1 Why this technology?

Homomorphic encryption checks every box for a Strange Tech. As we'll see later on, it has robust mathematical foundations with a long history, draws considerable attention in publications and news without being part of the mainstream, and isn't a commonly known technology. On top of that, the idea of "computing without seeing the data" is fascinating in its own right, and draws both engineers and investors alike.

Qualitatively, it's a great choice as a Strange Tech: (1) it has real empirical traction — Microsoft SEAL, Zama Concrete, and IBM's libraries are all available; (2) it is *strange* even to seasoned cryptographers — the idea of operating on encrypted data defies intuition; (3) it passes the Lindy filter with a solid theoretical and practical foundation built up over a long stretch of time — Rivest posed the question in 1978, Gentry solved it in 2009, and the lattices underpinning it trace back to the geometry of numbers in the 19th century; (4) its mathematical core — polynomials in quotient rings and controlled noise — can be learned at a basic level in a reasonable amount

DRAFT

of time, as this chapter will show; (5) it shares the lattice foundation with post-quantum cryptography, which makes it doubly interesting; and (6) it isn't the market's main bet — the mainstream talks about confidential computing with hardware enclaves or differential privacy, not homomorphic schemes.

1.2 A brief history

The name *homomorphic* comes from the Greek *homo* (same) and *morphḗ* (form): an encryption that preserves the form of operations, so that what is done with the encrypted data produces the same result as if it were done with the plaintext. The best way to understand it is with an analogy (Gentry 2010): imagine that Alice, a very distrustful jeweler, puts her raw materials inside a transparent glove box and locks it. She sends the box to a worker who can manipulate the pieces through the gloves — cut them, solder them, polish them — but without ever touching them directly. When he's done, Alice recovers the box and opens it with her key: inside she finds the finished ring. The worker did all the work without ever having had access to the materials. The question is whether such a thing is possible with data and the associated computations.

In 1978, Rivest, Adleman, and Dertouzos posed exactly that question mathematically: can computations be performed on encrypted data without decrypting it (Rivest, Adleman, and Dertouzos 1978)? For three decades the practical answer was "no." The existing schemes — RSA, ElGamal, Paillier — supported *one* operation (multiplication or addition), but not both; they are called *partially* homomorphic.

In 2009, Craig Gentry solved the open problem in his doctoral thesis, constructing the first *fully* homomorphic encryption scheme (FHE) based on ideal lattices (Gentry 2009). The key to his construction is *bootstrapping*: a "somewhat homomorphic" scheme evaluates its own decryption circuit, which refreshes the ciphertext's noise — the imprecisions that accumulate with each operation — and enables unlimited operations.

Since then, successive generations of schemes — BGV (2012), BFV (2012), GSW (2013), CKKS (2017), TFHE (2018) — have reduced the computational cost by several orders of magnitude, making the first real-world deployments viable.

In the following chapters we will first see how homomorphic encryption can be used in practice, then the possible applications and limitations of the technology (Chapter 4), and finally, the mathematical foundations of these

DRAFT

operations (Chapter 5), all interspersed with questions of varying difficulty to make sure we're understanding the concepts.

DRAFT

DRAFT

Chapter 2

Hands on

Daniel, no matter how good an engineer he is, doesn't know where to start when experimenting with a new technology. Building a prototype is one thing; building a production system with a service level suited to the application is a very different matter. But one way to get a feel for the basic intuitions of a technology is to start experimenting with it. Trying to understand the underlying math without any intuition about how it works tends to be almost impossible.

Before diving into the theory, let's see FHE in action with six simple recipes: we'll start by computing a mean over encrypted data, move on to variance — which requires multiplications — attempt a linear regression, hit the noise wall, break through it with *bootstrapping*, and close with an encrypted search where the server answers without knowing what it was asked. Each recipe poses a small problem and solves it step by step. By the end of this section you'll know what FHE can and cannot do *today*, before understanding the *why*.

All recipes use Python with TenSEAL, a library that implements the CKKS scheme — best suited for real-number arithmetic — except Recipe 5, which uses OpenFHE because TenSEAL does not expose *bootstrapping*.

The first step, common to all recipes, is to create the *cryptographic context* — the equivalent of preparing the "glove box" from the analogy. Each parameter has a reason that we'll see in Chapter 5:

```
import tenseal as ts

contexto = ts.context(
```

DRAFT

```
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60],
)
contexto.global_scale = 2**40
contexto.generate_galois_keys()
```

For now, here's what you need to know:

- `CKKS` is the scheme for operating on approximate real numbers. Other schemes work with integers or bits — see Table 5.3.
- `poly_modulus_degree = 8192` sets the degree N of the polynomial underlying the scheme. N is not chosen based on how much data we have, but on security requirements, circuit depth (how many chained multiplications we need), and the desired degree of parallelism. A larger N gives more security and more parallelism — each ciphertext can hold up to $N_{\text{slot}} = N/2$ independent values, called *slots* — but each operation is proportionally slower. With $N = 8192$ we have $N_{\text{slot}} = 4096$ slots available, although our examples use far fewer.
- `coeff_mod_bit_sizes = [60, 40, 40, 60]` determines how many chained multiplications can be performed before the accumulated noise destroys the result. Here, the two intermediate 40-bit values give two levels of multiplication — see Section 5.7.
- `global_scale = 2**40` sets the precision with which real numbers are represented inside the ciphertext. A larger scale means less approximation noise — but less room remains for chained multiplications. With 2^{40}, the absolute error for a simple operation like computing the mean is on the order of 10^{-7}, constant regardless of whether the operands are thousands or decimals.
- `generate_galois_keys()` generates the keys needed for *rotations* — operations that combine values from different *slots* within the ciphertext. Without them, `sum()` would fail. They are generated explicitly because they occupy tens of megabytes (about 32 MB with these parameters) — see Section 5.3.

With this context ready, let's look at the first recipe.

2.1 Recipe 1: Encrypted mean

Daniel decides to start with the simplest case: computing the mean salary over encrypted data and verifying that the result matches the plaintext calculation. We encrypt the data and, from this point on, all operations

DRAFT

run on the ciphertext — the plaintext data is no longer involved:

```
salarios = [3200, 4100, 2800, 5500, 3900]
salarios_cifrados = ts.ckks_vector(contexto, salarios)

# In a real deployment, this would run in the cloud,
# with no access to the plaintext data
# (CKKSVector does not implement division `/`,
# only multiplication `*`, so dividing by n is expressed
# as a multiplication by the constant 1/n)
media_cifrada = salarios_cifrados.sum() * (1 / len(salarios))
```

The five salaries are packed into a single ciphertext thanks to *packing* (Section 5.3), which with our configuration could pack up to $N_{\text{slot}} = N/2 = 4096$ values. The `sum()` operation uses internal rotations to add all the *slots*, and the division by 5 is a multiplication by a constant (1/5), cheap in FHE.

Finally, we decrypt the result:

```
media = media_cifrada.decrypt()[0]
media_claro = sum(salarios) / len(salarios)
print(f"Media (FHE):   {media:.2f} €")        # 3900.00 €
print(f"Media (claro): {media_claro:.2f} €")  # 3900.00 €
```

The result is practically identical — the absolute difference is on the order of 10^{-7}, inherent noise from the CKKS scheme that remains constant regardless of the operands' magnitude (we'll see why in Section 5.6), along with the execution time. Our example runs locally, but the operations on the ciphertext would be identical in a real deployment: the cloud provider would have calculated the mean salary without ever seeing a single actual number. Exactly what Elena needed.

The context parameters look arbitrary at first glance — why 8192 and not 4096? Why `[60, 40, 40, 60]`? Why 2^{40}? Each choice has a precise mathematical reason that we'll see in Chapter 5. But first, the following recipes show what FHE makes possible — and what it doesn't.

DRAFT

Review Questions

2.1. Ask an LLM: *"What's the difference between the BFV and CKKS schemes? When would you use each one?"*

2.2. What happens if you increase `poly_modulus_degree` to 16384? And if you reduce it to 4096? Experiment with the execution time.

2.2 Recipe 2: Encrypted variance

Daniel sends the mean to Elena, but she wants more: *"I need the spread — are the salaries homogeneous or are there outliers?"* The population variance is $\text{Var} = E[X^2] - (E[X])^2$, and computing it over encrypted data forces us to use the most expensive operation in FHE: multiplication between ciphertexts.

There's a subtle detail that deserves attention. The *naive* translation of the formula would be:

```
#  This does NOT work correctly in CKKS
ct = ts.ckks_vector(contexto, salarios)

cuadrados = ct * ct                          # cipher × cipher
media_cuadrados = cuadrados.sum() * (1/n)    # × constant

media = ct.sum() * (1/n)                     # × constant
media_al_cuadrado = media * media            # cipher × cipher

varianza = media_cuadrados - media_al_cuadrado  # → garbage
```

The code is mathematically correct, but produces a completely wrong result (in our example, a *negative* variance). Why? Every multiplication — between ciphertexts or by a constant — is followed by a *rescaling* that consumes a 40-bit link from `coeff_mod_bit_sizes` (Section 5.6 and Section 5.7 explain why). With `[60, 40, 40, 60]` there are only two such links — two multiplications' worth of headroom — before reaching the last link in the chain: the first value, 60 bits, the only one that survives once the rest are gone.

Let's see which link each branch reaches in this example:

Branch $E[X^2]$: `ct * ct` (consumes 1 link) → `.sum()` (free) → `* (1/n)` (consumes 1 link) → reaches the last link.

Branch $(E[X])^2$**:** `ct.sum()` (free) → `* (1/n)` (consumes 1 link) → `media * media` (consumes 1 link) → also reaches the last link.

Both branches consume the two available links and reach the same point in the chain — in fact, instrumenting the code shows they end up with the *same* scale and the *same* internal parameters (identical `scale` and `parms_id`): there's no mismatch between branches. The problem is something else, and subtler. At that last link, only the first modulus of the chain remains (60 bits), and with a scale $\Delta = 2^{40}$ that leaves just $60 - 40 = 20$ bits — a bit over a million — to represent the integer part of the result. But both $E[X^2]$ and $(E[X])^2$ here are around 15-16 million, far above that margin: the value silently overflows modulo that last link, and neither branch gives the correct result, even though both arrive with identical metadata.

The fix isn't to reorder the operations — reordering doesn't change how many links get consumed, and the overflow persists just the same — but to never exhaust that last link. If we remove the multiplication by a constant, moving the division by n to plaintext *before* encrypting, each branch goes from two multiplications (2 links) to just one (1 link), always leaving the 40-bit link spare in addition to the first one:

```
ct = ts.ckks_vector(contexto, salarios)
norm = [x / len(salarios) for x in salarios]
ct_n = ts.ckks_vector(contexto, norm)

# E[X²] = sum(x · x/n): one ct×ct mult (level 1)
media_cuadrados = (ct * ct_n).sum()

# (E[X])² = (sum(x/n))²: one ct×ct mult (level 1)
media = ct_n.sum()
media_al_cuadrado = media * media

# Both at the same level and scale - the subtraction works
varianza_cifrada = media_cuadrados - media_al_cuadrado
print(f"Varianza (FHE): {varianza_cifrada.decrypt()[0]:.2f}")
```

Let's confirm the result, branch by branch:

Branch $E[X^2]$**:** `ct * ct_n` (single multiplication, between two fresh ciphertexts) → `.sum()` (free) → stops one link short of the bottom of the chain.

Branch $(E[X])^2$**:** `ct_n.sum()` (free) → `media * media` (single multiplication) → also stops one link short of the bottom.

Each branch now consumes 1 link instead of 2, stopping one step above the bottom of the chain: instead of being left with only the first 60-bit modulus, we also keep an extra 40-bit link, multiplying the available margin by a factor of 2^{40} — more than enough to represent a few million without overflowing. That's why the final subtraction gives the correct result.

This is a real example of what makes FHE programming hard: it's not enough to translate a mathematically correct formula into code. Every multiplication — between ciphertexts or by a constant — consumes part of the level budget, and exhausting it entirely can leave the result without enough precision margin, even when both branches arrive with identical internal parameters. The programmer must design the circuit thinking not just about *what* it computes, but about how much precision margin the final result has in the worst case — and always leave one link to spare.

Review Questions

2.3. In Recipe 2, why does the formula $E[X^2] - (E[X])^2$ require two multiplications between ciphertexts while the alternative formula $\frac{1}{n}\sum(x_i - \bar{x})^2$ might require more? Which is more efficient in FHE?

2.4. Elena comes back with a new request: instead of the variance of a single vector, she now needs the **covariance** between two private encrypted vectors X and Y: $\mathrm{Cov}(X, Y) = E[XY] - E[X]E[Y]$. How many links would the implementation consume?

2.3 Recipe 3: Encrypted linear regression

Elena, as always, never stops and now needs something more ambitious: a predictive model that assigns a risk score to each patient. *"Can you have it ready for this afternoon? It's super urgent and we're drowning in work. The vendor has an already-trained linear regression model (the weights are public), but the patient data is confidential."*

Fortunately, FHE allows exactly this: encrypt the patient data, multiply by the plaintext weights, and sum — a homomorphic dot product.

```
# Patient data (encrypted) - the vendor cannot see it
features = [1.2, 0.7, 3.1, 0.4, 2.8]
features_cifrados = ts.ckks_vector(contexto, features)

# Model weights (plaintext) - not confidential
```

```
pesos = [0.5, -1.2, 0.8, 0.3, -0.6]

# Prediction: homomorphic dot product
prediccion = (features_cifrados * pesos).sum()
print(f"Riesgo (FHE): {prediccion.decrypt()[0]:.4f}")
```

Note that `features_cifrados * pesos` is a multiplication by plaintext constants, cheaper than a multiplication between ciphertexts because it requires no relinearization, although it does consume a level because of the *rescaling* (Section 5.7). For its part, `sum()` only uses rotations, which consume no levels, to do the computation efficiently with the Galois keys. The same context we already created would serve for a model with thousands of features — thanks to *packing* (Section 5.3), the $N_{\text{slot}} = 4096$ slots of the ciphertext would process them all in parallel. This is the basis of real-world deployments of *ML on encrypted data* that we'll see in the applications section (Chapter 3).

Review Questions

2.5. In the linear regression recipe, why is the multiplication `features_cifrados * pesos` cheaper than a multiplication between ciphertexts? What would happen if the weights were also encrypted?

2.6. If the model had 10,000 features and $N_{\text{slot}} = 4096$, how many ciphertexts would you need for a prediction? How would you combine the partial results?

2.4 Recipe 4: The noise wall

After a hellish week, Daniel is shutting down his office computer when he receives a desperate call from Elena at five in the afternoon — always at 5 or 6, especially on Fridays. *"Daniel! It's not working! The circle of death again! And also it says numbers overflow? Numbers overflow?! Where do they even go? Look at the face they made when I showed them the version you said worked! Didn't you test it?".*

"Yes, of course, before installing it we always test it."

"Well, I don't know what could have gone wrong. Since you were out during lunch, I asked Gabriel to add an extra correction factor."

"Is it possible he added a multiplication to the final result?".

"But what difference does that make? I don't know how he messed up…"

What happens if we ask for more operations than the modulus chain allows? Let's create a context with a single multiplicative level and attempt two chained multiplications:

```
ctx_1nivel = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 60],  # only 1 level
)
ctx_1nivel.global_scale = 2**40
ctx_1nivel.generate_galois_keys()

datos = ts.ckks_vector(ctx_1nivel, [3.0, 7.0, 2.0])

resultado = datos * datos   # OK - consumes the only level
print(resultado.decrypt())  # [9.0, 49.0, 4.0]

resultado = resultado * resultado  # FAILS - no levels left
# → Error: scale out of bounds
```

The library throws an exception on the second multiplication: it asked for a second link when there was only one, and TenSEAL refuses to continue rather than return a meaningless result. This is a deliberate design decision — but, as we saw in Recipe 2, it's not a universal guarantee: that explicit error only shows up when *more* budget is requested than is available. If instead the computation lands exactly at the last link, with the precision margin stretched to its limit — as happened to Daniel with the variance —, CKKS doesn't detect anything wrong and can return garbage with all the appearance of a valid result. The practical lesson is the same in both cases, and it is never to push the level budget all the way to the edge.

This wall is the practical manifestation of everything we'll see in Section 5.6 and Section 5.7, since each 40-bit level in `coeff_mod_bit_sizes` is a link that gets consumed with each multiplication. When they run out, there's no way forward — unless you resort to *bootstrapping*, which restores the levels at enormous cost (see Section 2.5 for a practical example and Chapter 5 for the theoretical explanation).

Review Questions

2.7. If you need to compute a degree-3 polynomial over encrypted data ($ax^3 + bx^2 + cx + d$), how many multiplicative levels do you need? What `coeff_mod_bit_sizes` would you configure?

2.8. Why does the library throw an error instead of returning an approximate result? What consequences would there be if it silently produced garbage in a medical or financial system?

2.5 Recipe 5: Breaking through the wall

Recipe 4 ended with an error — we ran out of multiplicative levels. But *bootstrapping* can restore them and let us keep operating. TenSEAL does not expose this functionality, so for this recipe we use OpenFHE, another reference library that does implement bootstrapping in CKKS.

The first step is to configure a CKKS context with bootstrapping enabled. This requires extra levels to evaluate its own internal circuit. To keep the demo fast, we use a small ring ($N = 4096$) with security validation disabled — with `HEStd_128_classic` (128-bit security against classical computers, the usual standard), OpenFHE would require $N = 2^{16}$ to support the bootstrapping depth and keep security at the established level. More demanding levels exist (192, 256 bits) and quantum variants that assume quantum computer attacks and require even larger N:

```
import openfhe as fhe

params = fhe.CCParamsCKKSRNS()
params.SetSecretKeyDist(fhe.SecretKeyDist.UNIFORM_TERNARY)
# Small ring for a fast demo; in production use
# HEStd_128_classic with a larger N.
params.SetSecurityLevel(fhe.SecurityLevel.HEStd_NotSet)
params.SetRingDim(1 << 12)

# Bootstrapping consumes its own levels to evaluate
# the decryption circuit homomorphically.
# level_budget = [collapsing_levels, raising_levels]
level_budget = [2, 2]
niveles_utiles = 2
profundidad = niveles_utiles + fhe.FHECKKSRNS.GetBootstrapDepth(
    level_budget, fhe.SecretKeyDist.UNIFORM_TERNARY
)
```

```
params.SetMultiplicativeDepth(profundidad)
params.SetScalingModSize(48)
params.SetFirstModSize(60)
num_slots = 4  # slots occupied in the ciphertext
params.SetBatchSize(num_slots)

cc = fhe.GenCryptoContext(params)
cc.Enable(fhe.PKESchemeFeature.PKE)
cc.Enable(fhe.PKESchemeFeature.KEYSWITCH)
cc.Enable(fhe.PKESchemeFeature.LEVELEDSHE)
cc.Enable(fhe.PKESchemeFeature.ADVANCEDSHE)
cc.Enable(fhe.PKESchemeFeature.FHE)

keys = cc.KeyGen()
cc.EvalMultKeyGen(keys.secretKey)
cc.EvalBootstrapSetup(level_budget, [0, 0], num_slots)
cc.EvalBootstrapKeyGen(keys.secretKey, num_slots)
```

OpenFHE's parameterization is different from TenSEAL's, but the concepts are the same. In TenSEAL we specified the full modulus chain as a list of bit sizes: `coeff_mod_bit_sizes=[60, 40, 40, 60]` — two intermediate levels of 40 bits each. OpenFHE separates that information: `SetMultiplicativeDepth` indicates *how many* levels there are (an integer), and `SetScalingModSize(48)` indicates *how many bits* each intermediate level has. `SetFirstModSize(60)` sets the size of the first link, just like the initial 60 in TenSEAL.

`SetBatchSize(num_slots)` indicates how many *slots* the ciphertext occupies. This is needed so the context correctly aligns the bootstrapping's internal rotations with the ciphertext length.

The novelty here is `level_budget` and `GetBootstrapDepth`. Bootstrapping consumes its own multiplicative levels when internally evaluating the decryption circuit; `level_budget = [2, 2]` indicates how many levels to dedicate to each of its two internal phases — we'll see what these phases do in Section 5.8. `GetBootstrapDepth` takes these budgets and calculates how many extra levels the bootstrapping circuit needs in total, which are added to the programmer's `niveles_utiles`.

`EvalBootstrapSetup(level_budget, [0, 0], num_slots)` prepares bootstrapping internally. `num_slots` must match the earlier `SetBatchSize`. `EvalBootstrapKeyGen` generates the rotation keys needed for bootstrapping.

Finally, we enable `PKESchemeFeature.FHE` (full FHE, not just *leveled*) — without this, `EvalBootstrap` would not be available.

Now we encrypt the same data from Recipe 4 and repeat the experiment:

```
datos = [3.0, 7.0, 2.0, 5.0]
ptxt = cc.MakeCKKSPackedPlaintext(datos)
ctxt = cc.Encrypt(keys.publicKey, ptxt)

ctxt = cc.EvalMult(ctxt, ctxt)   # x² - consumes 1 level
ctxt = cc.EvalMult(ctxt, ctxt)   # x  - consumes another level

# Levels exhausted. In Recipe 4 this was the end.
# Here: bootstrapping to the rescue.
ctxt = cc.EvalBootstrap(ctxt)

# Now we can keep operating!
ctxt = cc.EvalMult(ctxt, ctxt)   # x

result = cc.Decrypt(ctxt, keys.secretKey)
result.SetLength(len(datos))
# → [6561.0, 5764801.0, 256.0, 390625.0]  (3 , 7 , 2 , 5 )
```

The call to `EvalBootstrap` is where the magic happens: it takes the exhausted ciphertext, homomorphically evaluates the decryption circuit (the process we'll detail in Chapter 5), and produces a fresh ciphertext with restored levels. The operation takes on the order of one second — compared to the milliseconds of a normal multiplication — and the resulting ciphertext carries somewhat more approximation error. But it works: the multiplication that in Recipe 4 threw "scale out of bounds" now completes without problems, and we can even chain a third one.

DRAFT

Review Questions

2.9. Bootstrapping restores multiplicative levels but takes ~1 s compared to ~1 ms for a normal multiplication. If you need to evaluate a circuit of depth 20, which strategy is better: dimensioning 20 levels from the start, or using 5 levels with bootstrapping every 5 multiplications?

2.10. Ask an LLM: *"What is circular security in FHE and why is it necessary for bootstrapping?"*

2.11. Why do you think TenSEAL doesn't expose bootstrapping directly, unlike OpenFHE? What trade-off have TenSEAL's designers faced?

2.6 Recipe 6: Encrypted search

Elena is satisfied with the risk models, but now she has another need: *"I need to look up the score of a specific patient in the vendor's database. But I don't want them to know which patients I'm interested in — that would reveal clinical information."*

This is a *Private Information Retrieval* problem: retrieving a record from a database without the server knowing which one was requested. With FHE, the solution is surprisingly simple.

The trick is to encrypt a *one-hot* vector — a vector of zeros with a single 1 in the position of the desired record. The server multiplies that encrypted vector by its database (in plaintext) and sums all the products. Since multiplying by an encrypted 0 "nullifies" all positions except the queried one, the encrypted result contains only the sought value. The server has processed *all* the records, but only one survived the multiplication — and the server doesn't know which.

```
# Vendor's database (plaintext on their server)
scores = [85.2, 62.0, 91.7, 45.3, 78.9, 33.1, 70.4, 56.8]

# Elena wants the score of the patient at position 3,
# without the vendor knowing it's patient 3
indice = 3
consulta = [1.0 if i == indice else 0.0
            for i in range(len(scores))]
consulta_cifrada = ts.ckks_vector(contexto, consulta)
```

```
# The server executes this - it only sees the ciphertext
resultado_cifrado = (consulta_cifrada * scores).sum()

# Elena decrypts
print(resultado_cifrado.decrypt()[0])  # → 45.3
```

The operation `consulta_cifrada * scores` is a multiplication by constants, cheap but consuming one level because of the rescaling, just like in Recipe 3. And `sum()` uses rotations, just like in Recipe 1. We don't need any new operations; we just combine the primitives we already know in a different way. This is a pattern that appears over and over in FHE: the same two basic operations (multiplication and addition) give rise to very different applications depending on how the circuit is designed.

But there's an inescapable cost: **the server must process all records for each query**. It can't "skip" the positions that are zero, because it doesn't know which ones they are — the vector is encrypted. If the database has 10,000 records, the server multiplies and sums all 10,000, even though only one contains useful information. That's the price of privacy: the query is hidden precisely because it's indistinguishable from any other. If the server could process only a subset, it could deduce by elimination which position was queried.

With $N = 8192$, each ciphertext stores up to $N_{\text{slot}} = 4096$ slots — enough for a database of that size without needing multiple ciphertexts. For larger databases, the records need to be split into blocks of N_{slot}, sending one query ciphertext per block and summing the partial results. Who does that splitting depends on the library: OpenFHE, closer to the mathematical scheme, rejects with an error any vector that exceeds N_{slot} and leaves the chunking entirely up to the programmer; TenSEAL, more application-oriented, detects it and resolves it internally — creating several ciphertexts and spreading the final sum across them —, though that convenience has a limit: operations like matrix multiplication stop being available as soon as the vector spans more than one ciphertext.

Does this scale to databases with millions of records? Not particularly well, and it's not just an engineering matter — it's a fundamental limitation of the scheme itself. Splitting the records into more blocks doesn't reduce the server's total work, it only redistributes it — the cost still grows linearly with the size of the database, because each additional block requires its own multiplication and sum over N_{slot} records. With 10,000 records the delay is negligible; with 10 million, each query can take minutes even with the hardware accelerations we'll see in Chapter 4. No amount of SIMD

DRAFT

packing changes this trend: it only reduces the constant, not the order of growth. This is the price of the server being unable to discard any record without risking a leak of which one was queried. In practice, large-scale private-retrieval systems mitigate the problem by partitioning the database into public categories (say, by hospital or by year), narrowing the pool of candidate records before applying this technique within each partition — at the cost of revealing that category, which is assumed not to be sensitive.

Review Questions

2.12. In the encrypted search, why must the server process *all* records to answer a single query? What information would leak if it processed only a subset?

2.13. If the vendor's database has 10,000 records and $N_{\text{slot}} = 4096$, how many ciphertexts does the client need for a query? And how many must the server process?

2.14. Why doesn't splitting the database into more blocks of N_{slot} reduce the server's total cost? What privacy trade-off do private-retrieval systems that partition by public categories accept?

DRAFT

Chapter 3

Applications

Elena's case — analyzing medical data in the cloud without exposing it — is not a theoretical exercise. It's exactly the kind of problem that has driven investment in FHE over the past decade: outsourcing infrastructure and software to third parties, even in sectors as sensitive as healthcare or national security, demands keeping data confidential across every layer of the system — infrastructure and software as well as data in transit. Let's look at the scenarios where homomorphic encryption delivers the most value.

Healthcare and clinical data. A hospital wants to run predictive diagnostic models on its patients' medical records using a cloud provider's infrastructure. Regulations (HIPAA in the U.S., GDPR in Europe) prohibit sending medical data in plaintext to third parties. The usual alternatives are to anonymize the data or add statistical noise via differential privacy (the chapter on differential privacy). Anonymization is fragile: multiple studies have demonstrated re-identification of medical records by cross-referencing apparently innocuous variables. Differential privacy, for its part, degrades result accuracy by injecting noise, which can be unacceptable for individual diagnoses. With FHE, the hospital encrypts the data before uploading it to the cloud, the provider runs its models on the ciphertexts, and returns an encrypted result that only the hospital can decrypt. The provider never sees a single diagnosis.

Regulated financial analysis. Bank fraud detection is more effective when multiple institutions share patterns of suspicious transactions. But sharing financial data between banks clashes head-on with privacy regulations and competitive reluctance. Today the approach relies on confiden-

DRAFT

tiality agreements and controlled environments, but the receiving bank still sees the data in plaintext. FHE allows each bank to encrypt its transactions so that a centralized service can run detection algorithms on all the encrypted data, without any entity accessing the others' records.

Machine learning on sensitive data. Training a machine learning model requires access to large volumes of data, which often belong to different organizations — hospitals, insurers, government agencies. FHE allows multiple parties to contribute encrypted data to a joint training process without revealing individual information. This opens the door to collaborations between organizations that are currently infeasible due to legal or trust barriers.

Encrypted searches. A user searches for information on a web service, but doesn't want the provider to know their queries — think searches for medical symptoms or legal advice. With FHE, the query is sent encrypted, the server processes it against its index without decrypting it, and returns encrypted results that only the user can read. The provider offers the service without knowing what was searched.

Secure computation outsourcing. Beyond the above sectors, any organization that outsources computation to the cloud faces the same dilemma: trust the provider or don't use the cloud. FHE eliminates that trade-off by allowing data to remain encrypted throughout processing. The provider never has access to plaintext data, and it can also be combined with zero-knowledge proofs (Recommended Readings) so that the provider proves it executed the computation correctly — without revealing anything about the data or the process.

Review Questions

3.1. For the healthcare, finance, and machine learning applications, what alternative is used in practice today and why is it unsatisfactory?

3.2. Ask an LLM: *"Which companies or libraries offer homomorphic encryption in production today? What is the state of maturity?"* Verify the answers with a web search.

DRAFT

Chapter 4

Current limitations

We've seen that FHE works and has real applications. But between the concept and widespread deployment there's a series of obstacles worth understanding precisely. The limitations are ordered from highest to lowest practical impact — from those that most restrict what products can be built and how they're programmed, to those that are mitigable with complementary techniques.

Performance. An individual operation on ciphertexts is between 10^4 and 10^6 times slower than its plaintext equivalent. An integer multiplication that a modern processor executes in nanoseconds requires milliseconds in FHE, because it involves multiplying polynomials of degree N via the NTT — a fast transform that accelerates polynomial arithmetic (Chapter 5) — applying *relinearization* — a step that keeps the ciphertext in manageable format after each multiplication — and executing *rescaling* — the scale adjustment that controls error growth (Section 5.6) — all on coefficients of hundreds of bits. With $N = 8192$, each elementary operation manipulates hundreds of kilobytes of algebraic data. Three avenues reduce this gap. First, *packing* (Section 5.3) amortizes the cost by processing up to $N/2$ values in parallel per operation — the most important performance gain in practice. Second, processor SIMD instructions (AVX-512 via Intel HEXL) and GPUs accelerate the NTT and modular arithmetic, contributing between 2× and 100× depending on the operation. Third, dedicated hardware accelerators (FPGAs and ASICs, like the DARPA DPRIVE projects) aim for additional 10^3–10^4× gains, which would close the gap to 10–100×. Specialized compilers (Zama Concrete, Google FHE transpiler) complement these three avenues by automatically optimizing the arithmetic

DRAFT

circuit. Even with all these improvements, the cost remains prohibitive for low-latency applications — but is starting to be viable for *batch* workloads where privacy justifies the overhead.

Only additions and multiplications. The underlying algebraic scheme — called RLWE, which we'll see in Chapter 5 — provides only two native operations: polynomial addition and multiplication. Any other function — comparisons, sorting, conditional branches, nonlinear functions like cosine or sigmoid — must be *approximated* by a high-degree polynomial, and each multiplication of that polynomial consumes a level from the budget. A simple `if (x > 0)` requires evaluating a polynomial of degree 10 to 30, consuming between 4 and 10 multiplicative levels. This means that algorithms that are trivial in plaintext — like finding the maximum of a list — become circuit design problems: finding the maximum of n encrypted values requires a pairwise-comparison tournament, with $\lceil \log_2 n \rceil$ rounds to reduce the list to a single winner, and each round evaluates the sign polynomial over every pair — between 4 and 10 levels per round. With 1000 values that's 10 rounds and between 40 and 100 levels total, well above the typical budget of 10 to 30 levels without resorting to the *bootstrapping* we'll cover further down in this same section. The designer must then choose between interleaving *bootstrappings* — at ~1 second each — or switching to TFHE: a scheme built on boolean logic circuits that represents data bit by bit and evaluates comparisons as sequences of logic gates, without polynomial approximation, at the cost of losing the SIMD parallelism that makes CKKS worthwhile for bulk arithmetic (Chapter 5).

Programming complexity. Programming on encrypted data is nothing like programming in the clear. The developer must design the *arithmetic circuit* — the complete sequence of additions and multiplications the scheme must execute — before starting: plan how many chained multiplications are needed, choose cryptographic parameters based on that depth, pack the data to maximize SIMD parallelism, and give up any data-dependent branching (you can't inspect an encrypted value to decide which branch to take). Debugging is opaque: intermediate ciphertexts can't be "printed" without decrypting them, which defeats the purpose of privacy. But perhaps the trickiest aspect — because it produces no visible errors — is *scale management*, a problem specific to CKKS. Exact arithmetic schemes (BFV, BGV) also require level management, but a mismatch either causes an explicit error or is absorbed automatically by the library — it never silently corrupts the result. In CKKS, however, the approximate encoding of real numbers introduces a scale factor Δ that gets transformed with each operation, so a mathematically correct formula translation can produce nonsensical results if the computation exhausts the precision margin

DRAFT

available at the last link of the modulus chain — as we saw in Recipe 2 with the encrypted variance, where both branches arrived with identical scales and internal parameters and still overflowed — or if different branches end up with genuinely incompatible scales. In neither case does CKKS throw an error: it simply misinterprets the internal bits and returns garbage. The programmer must manually track how each operation — ciphertext×ciphertext multiplication, multiplication by constant, rescaling — transforms the ciphertext's scale Δ, and ensure that all branches arrive at the same state before being combined. Compilers like Concrete (Zama) and Google's FHE transpiler automate part of this process, transforming high-level code into optimized FHE circuits, but the compilation technology is still in an early stage.

Limited circuit depth. The multiplicative level budget is fixed *before* encrypting, via the `coeff_mod_bit_sizes` modulus chain (Section 5.7). If the circuit needs more multiplications than budgeted, you must resize the parameters (increasing N and q, which makes *all* operations more expensive) or resort to bootstrapping, which takes ~1 second and spends some of the newly restored levels. There's no way to "add levels on the fly" — the circuit topology must be known in advance.

Approximate results (CKKS). The CKKS scheme, the most widely used for real numbers, produces results with an inherent error on the order of 10^{-6} in typical configurations. For statistics and machine learning this is usually acceptable, but not for applications requiring exact arithmetic (accounting, voting). The BFV and BGV schemes offer exact integer arithmetic, but don't support real numbers directly. Additionally, CKKS has a particular security nuance: since decrypted results carry traces of the ciphertext's internal noise, an attacker who observes enough decryptions could extract information about the secret key. Current libraries implement mitigations — such as adding extra noise when decrypting — but the developer should keep a practical rule in mind: don't expose intermediate decryptions to third parties.

Ciphertext expansion. As we'll see when describing *RLWE* in Chapter 5, each ciphertext is internally a pair of polynomials with N coefficients modulo q, which means a size of about $2 \times N \times \lceil \log_2 q \rceil$ bits. With our parameters ($N = 8192$ and a ciphertext that, as we'll see in Section 5.7, lives modulo $q \approx 2^{140}$, split among three primes), and since in practice each prime takes a 64-bit word per coefficient, that gives $2 \times 8192 \times 3 \times 8$ bytes $\approx$ 400 KB. An 8-byte number becomes a ~400 KB ciphertext — a 50,000× expansion. *Packing* (Section 5.3) reduces the amortized cost to ~100 bytes per value, but the cryptographic keys are even more voluminous: Galois

DRAFT

keys occupy tens of megabytes (about 32 MB with $N = 8192$) and bootstrapping keys can reach several gigabytes. Transmitting these keys to a cloud provider can take minutes even with fast connections, and the server's RAM consumption multiplies.

Key sizes. The secret key is compact (a few kilobytes), but the cryptographic material needed for homomorphic operations is enormous. The Table 5.2 in Section 5.5 shows the four necessary keys; the relinearization and especially the Galois keys — one per rotation position you want to enable — are much larger than a ciphertext. With the parameters of our recipes ($N = 8192$), TenSEAL serializes the relinearization keys in a little over 1 MB and the Galois keys in about 32 MB; with $N = 16384$ and four levels, the latter already reach around 160 MB. If bootstrapping is added, which requires much larger rings and many more rotations, the evaluation keys reach gigabytes. This limits the devices where FHE can run (an IoT sensor can't store tens or hundreds of megabytes of keys) and adds latency in scenarios where keys must be transmitted to the server.

Malleability and lack of integrity. By definition, a homomorphic scheme allows anyone to transform a ciphertext into another valid ciphertext that encrypts a related message — that's precisely what makes it useful. But that same property means ciphertexts are *malleable*: an attacker could modify an encrypted result without being detected (for example, adding a constant to someone's encrypted salary). FHE schemes guarantee that an attacker cannot distinguish between two ciphertexts — they can't tell which message was encrypted. But they *don't* guarantee that an attacker can't *modify* a ciphertext in a useful way without being detected. In practice, computation integrity is protected by combining FHE with zero-knowledge proofs (Recommended Readings) or with verifiable computation schemes.

Review Questions

4.1. If homomorphic operations are 10^4–10^6 times slower, for what types of applications is the cost acceptable and for which is it not?

4.2. Ask an LLM: *"How does the performance of FHE in 2024 compare with 2015? What were the main advances?"*

4.3. An engineer needs to implement a comparison `x > y` on encrypted data with FHE. Why can't they do it directly? What alternative do they have and what is its cost?

DRAFT

Chapter 5

Foundations

Daniel has already had a chance to experiment with the technology — what he calls "messing around with it" — and for Elena the results are enough to move the project forward. The vendor gets its analysis, and legal gets its confidentiality. We've also seen that the code works and that the potential applications are real. But is this level of understanding enough to keep a new technology running in a production system? Daniel and his team will have to maintain and extend it whenever business asks, and explain, when something breaks — and something always does — the why, not just the how. Finally, to avoid falling behind, they will need to keep an eye on which innovations can have a real impact on performance, functionality, or cost. What do they need to know for all that?

How is it possible that a computer runs calculations on data it can't see, and why is it so expensive even with the latest advances and optimizations? Let's recall the analogy of the glove box for manipulating jewels or cooking without touching the ingredients: Alice puts her materials inside a locked box, and a worker manipulates them through the gloves without ever touching them. In FHE, the data is the materials, the encryption is the box, and the homomorphic operations are the gloves. Let's see how that "box" is built mathematically, and along the way we'll understand every parameter from the code we just ran. The key lies in a branch of mathematics that few people associate with cryptography: lattices.

DRAFT

5.1 Lattices

The security of homomorphic encryption schemes doesn't depend on integer factorization — like RSA — but on the difficulty of solving geometric problems in structures called *lattices*. Understanding these structures is what will allow Daniel to answer with confidence when legal asks if patient data is safe in the cloud.

And what is a lattice? A lattice is a discrete set of points in an N-dimensional space, generated by integer linear combinations of a set of basis vectors $B = \{b_1, \ldots, b_N\}$:

$$L(B) = \left\{ \sum_{i=1}^{N} a_i b_i : a_i \in \mathbb{Z} \right\}$$

In general, the basis vectors b_i can be real, but in cryptography we always work with integer vectors modulo q — that is, $b_i \in \mathbb{Z}_q^N$ — so all lattice points are also integers. Think of a regular grid as in Figure 5.1, but in hundreds of dimensions.

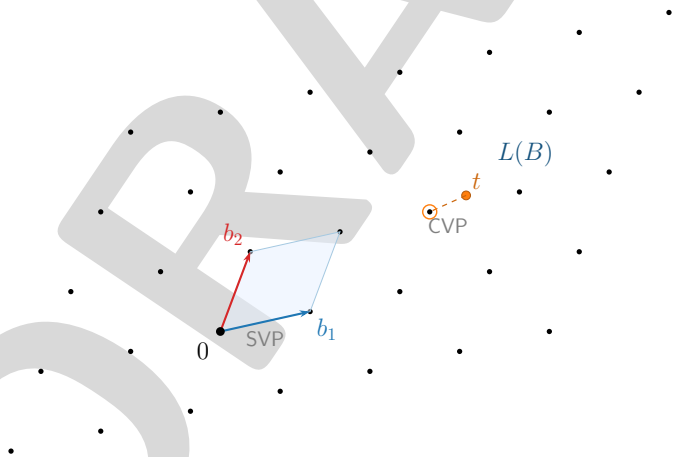


Figure 5.1: Two-dimensional lattice generated by two basis vectors b_1 and b_2. The shaded region is the *fundamental domain* (parallelogram defined by the basis). The figure illustrates two fundamental problems: finding the shortest nonzero vector (SVP, *Shortest Vector Problem*) and, for an external point t, finding the nearest lattice point (CVP, *Closest Vector Problem*). Both are computationally intractable in high dimensions.

DRAFT

Modern cryptography relies on two fundamental problems over these structures based on integer vector bases:

- **SVP** (*Shortest Vector Problem*): find the shortest nonzero vector in the lattice.
- **CVP** (*Closest Vector Problem*): given an arbitrary point in space, find the nearest lattice point. If the *exact* nearest point is required, the problem is NP-hard. In its approximate version it's no longer NP-hard, but the best known algorithms still have exponential cost in the lattice dimension N.

CVP is at least as hard as SVP, and both are the foundation of lattice cryptography. In cryptography, the approximate versions are used — relaxed enough not to be NP-hard, but strict enough that no known algorithm, classical or quantum, can solve them in reasonable time. At the typical FHE dimensions ($N \geq 1024$), the problem is intractable, and this difficulty is what protects the data. But a lattice by itself doesn't encrypt anything — we need to translate this geometric difficulty into a mechanism that allows encryption and operations.

Review Questions

5.1. What is the difference between SVP and CVP? Why is CVP at least as hard as SVP?

5.2. The security of FHE is based on the fact that lattice problems are hard "even for quantum computers." What distinguishes these problems from those used by RSA (factorization) against quantum attacks?

5.2 From lattices to polynomial rings: LWE and RLWE

With the geometric foundation in place, the next challenge is practical: how do we turn the hardness of finding points in a lattice into a usable, efficient encryption algorithm? That efficiency is what enables Daniel's code to compute Elena's average salary in milliseconds rather than hours. The first conceptual step is the *Learning With Errors* (LWE) scheme, which translates lattice geometry into linear algebra: vectors and matrices.

In LWE, the **private key** is a secret vector s of N integers. The **public key** is a pair (A, b) consisting of a random $N \times N$ integer matrix A and a vector b, which is computed as:

DRAFT

$$b = A \cdot s + e \pmod{q}$$

where e is a small noise vector. Put differently, b is the secret s "masked" by the matrix and the noise. Anyone can see A and b, but only someone who knows s can take advantage of the relationship between them. Without the noise e, recovering s from A and b would be freshman linear algebra (Gaussian elimination). But with the noise, b is a point *near* the lattice generated by the columns of A, though not exactly *on* it — and finding the nearest point is CVP. The noise turns a trivial problem into an intractable one.

Building on this structure, encrypting a message m (encoded as a vector) consists of adding it directly to the masked value:

$$c = A \cdot s + e + m \pmod{q}$$

Someone who knows s can recover the message m from the ciphertext c simply by subtracting $A \cdot s$:

$$m = c - (A \cdot s) \pmod{q} = m + e \pmod{q} \approx m \pmod{q}$$

LWE works, but has a performance problem: each operation on a ciphertext involves a matrix-vector product — $O(N^2)$ integer multiplications. With the dimensions needed for FHE ($N \geq 1024$), that's millions of operations per homomorphic addition or multiplication. Moreover, the matrix A has N^2 integers: with $N = 8192$, storing it requires more than 67 million integers. Too slow and heavy to be practical.

The solution is to replace vectors and matrices with **polynomials**. A polynomial of degree less than N — for example, $3x^2 + 5x + 1$ — is nothing more than a vector of N coefficients $(1, 5, 3, 0, \ldots, 0)$ written in a different notation. At first glance, it looks like we haven't gained anything, but if we restrict the polynomials to a specific *ring*, something remarkable happens.

But what is a ring, and what does it mean to restrict a polynomial to one? A ring is simply a set of elements — here, polynomials — together with two operations, addition and multiplication, that can be freely combined without ever leaving the set: adding or multiplying two elements of the ring always produces another element of the ring. $\mathbb{Z}_q$, the integers modulo q we already used in LWE, is the simplest example of a ring: adding or multiplying two integers modulo q gives another integer modulo q, never "escaping" the range $[0, q-1]$. Our polynomial ring needs that same closure

DRAFT

property, and in algebraic notation it's written $R_q = \mathbb{Z}_q[x]/(x^N + 1)$: polynomials with integer coefficients modulo q (i.e., $\mathbb{Z}_q[x]$, the same modular arithmetic as in LWE), with the additional restriction that $x^N + 1 = 0$, meaning $x^N = -1$. This restriction ensures the degree never grows: when multiplying two polynomials, if any term exceeds degree $N - 1$ — and would therefore fall outside the ring — the rule "folds" it back into the range $[0, N - 1]$, just as the modulo operation does with an integer.

Let's see an example with $N = 4$ and $q = 7$. We start with two polynomials in R_7:

$$a(x) = 2x^3 + x + 3, \qquad b(x) = x^3 + 4x^2 + 1$$

When multiplied, terms of degree greater than 3 appear. The ring's trick is to apply $x^4 = -1$: for example, $2x^3 \cdot x^3 = 2x^6 = 2 \cdot x^2 \cdot x^4 = -2x^2 = 5x^2 \pmod 7$. After folding all terms, the result is $2x^3 + 3x^2 + 2$ — always a polynomial with 4 coefficients, just like LWE vectors. The crucial point is that this restriction has exactly the same effect as restricting the set of possible matrices A to *negacyclic* matrices (whose rows are rotations of a single vector with alternating signs). That is, all the information in the matrix A is encoded in the coefficients of a single polynomial $a(x)$.

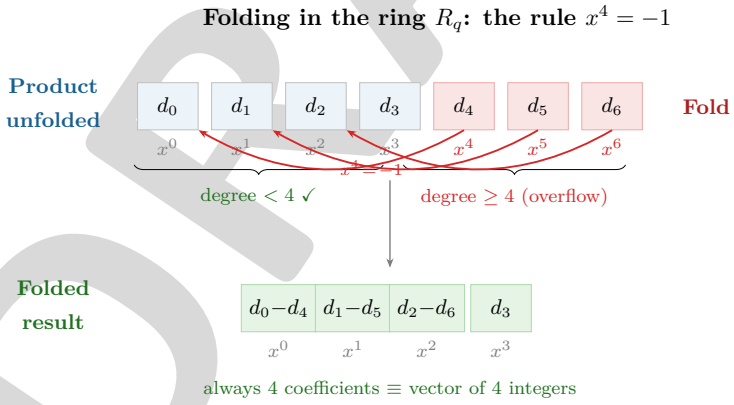


Figure 5.2: Folding in the ring R_q. When multiplying polynomials, terms of degree N or higher fold back to the range $[0, N-1]$ via the rule $x^N = -1$. This restriction is equivalent to working with negacyclic matrices, encoding an $N \times N$ integer matrix in just N coefficients.

DRAFT

The result is *Ring Learning With Errors* (RLWE): the version of LWE where vectors and matrices are replaced by polynomials in R_q. As before in LWE's algebraic scheme, a single equation — this time polynomial — encodes the relationship between the public key and the private key:

$$b(x) = a(x) \cdot s(x) + e(x) \pmod{q,\ x^N + 1}$$

The roles stay the same: the private key is the secret polynomial $s(x)$, the public key is the pair $(a(x), b(x))$, and the small error polynomial $e(x)$ is discarded after the key is generated.

The gain is twofold. First, in storage: the LWE matrix A has N^2 integers; the RLWE polynomial $a(x)$ has only N. With $N = 8192$, we go from 67 million integers to 8192 — an 8,000× reduction in key size. Second, in speed: multiplying a matrix by a vector costs $O(N^2)$; multiplying two polynomials in R_q costs only $O(N \log N)$ thanks to the *Number Theoretic Transform* (NTT), the modular version of the FFT (*Fast Fourier Transform*). With $N = 8192$, that's ~100,000 operations versus ~67 million — three orders of magnitude faster. This efficiency is what makes FHE viable.

i The NTT in five minutes

Multiplying two polynomials "by hand," the way we learned in school, means multiplying every coefficient of the first by every coefficient of the second: N^2 products. The NTT avoids that cost with the same trick as the FFT, built on a simple idea: a polynomial of degree less than N is completely determined by its values at N distinct points — just as two points determine a line. And once polynomials are represented by their values, multiplying them is trivial: the value of the product at each point is the product of the values, $c(x_k) = a(x_k) \cdot b(x_k)$ — only N multiplications. The method has three steps: evaluate both polynomials at the N points (the transform), multiply point by point, and convert back to coefficients (the inverse transform).

Evaluating a polynomial at N arbitrary points would again cost N^2 operations. The FFT sidesteps that problem by choosing special points that let it split the computation in half over and over — $\log_2 N$ times, with about N operations each time — so the full transform costs $O(N \log N)$. The NTT inherits exactly that algorithm, and with it its requirement: N must be a power of 2 (4096, 8192, 16384…), so it can be halved all the way down to 1.

The FFT uses the complex roots of unity as its points, with floating-

point arithmetic and therefore rounding errors that are unacceptable in cryptography. The NTT does exactly the same thing with integers modulo q, with no rounding at all. Moreover, for our ring the points chosen are the roots of x^N+1 modulo q: at those points x^N is exactly -1, so the ring's folding is applied automatically, at no extra cost. Those roots only exist if q is a prime of the form $q \equiv 1 \pmod{2N}$, which is what libraries pick internally when they build the modulus chain we'll see in Section 5.7.

A minimal example, with $N = 2$ and $q = 5$, lets us follow the method step by step. We want to multiply $a(x) = x + 2$ by $b(x) = 3x + 1$ in the ring $\mathbb{Z}_5[x]/(x^2 + 1)$:

1. **Choose the points.** We need the roots of $x^2 + 1$ modulo 5 — numbers whose square equals $-1 \equiv 4$. They are 2 and 3: $2^2 = 4$ and $3^2 = 9 \equiv 4 \pmod 5$.
2. **Transform (evaluate).** Substitute each point for x:
   - at $x = 2$: $a(2) = 2 + 2 = 4$ and $b(2) = 3 \cdot 2 + 1 = 7 \equiv 2$;
   - at $x = 3$: $a(3) = 3+2 = 5 \equiv 0$ and $b(3) = 3 \cdot 3+1 = 10 \equiv 0$.
3. **Multiply point by point.** At $x = 2$: $4 \cdot 2 = 8 \equiv 3$. At $x = 3$: $0 \cdot 0 = 0$. Two multiplications instead of the four of the schoolbook method.
4. **Convert back to coefficients (inverse transform).** We look for the polynomial $c(x) = c_1x + c_0$ with $c(2) = 3$ and $c(3) = 0$. Subtracting the two conditions, $c_1 = c(3) - c(2) = -3 \equiv 2$; solving for the other, $c_0 = c(2) - 2c_1 = 3 - 4 = -1 \equiv 4$. So $c(x) = 2x + 4$.
5. **Check the long way.** Multiplying by hand, $(x+2)(3x+1) = 3x^2+7x+2$. Folding with $x^2 = -1$ gives $-3+7x+2 = 7x-1 \equiv 2x + 4 \pmod 5$: the same result.

With $N = 2$ the savings are negligible, but with $N = 8192$ they make the difference between some 67 million products and about 100,000 operations. In practice, libraries keep ciphertexts almost always in their value representation (after step 2), so each multiplication boils down to step 3: just N products. The same idea — representing a polynomial by its evaluations at the roots of $x^N + 1$ — will reappear in the next section (Section 5.3), this time with complex roots, to pack thousands of values into a single ciphertext.

The name RLWE summarizes the mechanism. *Ring* refers to the polynomial ring — the source of efficiency. *Learning With Errors* describes the underlying problem: given $b = a \cdot s + e$, can one "learn" the secret s? Without the error, basic linear algebra would suffice. With it, recovering

DRAFT

s is equivalent to solving CVP in a lattice formed by polynomials — what we'll soon call an *ideal lattice* — exactly the difficulty from the previous subsection.

One nuance is worth making explicit: the jump from LWE to RLWE isn't free in security terms. LWE has a theoretical guarantee by being based on arbitrary lattices; RLWE, by contrast, only has that guarantee by being based on lattices restricted to the structure of the polynomial ring R_q (what's called *ideal lattices*). This extra degree of structure is precisely what enables the use of the NTT and its performance gain, but it could also, in principle, give an attacker a foothold that doesn't exist in the general case. In practice, no attack exploiting this structure is known for the ring families actually used (powers of two, such as $x^{8192} + 1$), so RLWE is treated as being as secure as LWE — but formally it rests on a distinct, more specific assumption.

CKKS, the scheme used in the hands-on section (Chapter 2), is built directly on RLWE, and the connection is deep. Unlike other schemes that treat the error e as something to be eliminated entirely, CKKS interprets it as a *precision error* — similar to what happens in floating-point arithmetic. Since RLWE already includes an inherent error, the scheme reuses it to represent real numbers as approximate values: the cryptographic noise and the approximation error are managed together through the *rescaling* techniques we'll see in Section 5.7.

With the mathematical structure in place — an efficient ring, noise that guarantees security, NTT that accelerates operations — what remains is deciding what to put inside it, that is, how to turn a vector of data into a polynomial of R_q. The NTT already gave us the hint: a polynomial can also be represented by its evaluations at the roots of $x^N + 1$, and that representation is what allows thousands of data values to be packed into a single polynomial.

DRAFT

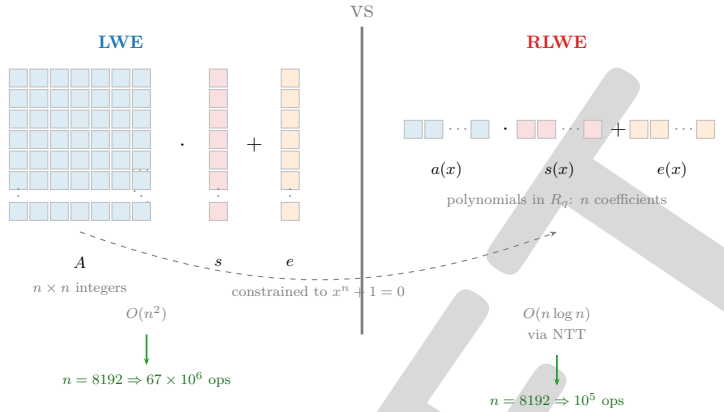


Figure 5.3: LWE versus RLWE. On the left, LWE requires an $N \times N$ integer matrix A. On the right, RLWE encodes the same information in a polynomial of N coefficients, reducing the cost from $O(N^2)$ to $O(N \log N)$ via the NTT.

Review Questions

5.3. In standard LWE, the public part is an $N \times N$ integer matrix A. In RLWE it is replaced by a single polynomial of N coefficients. What algebraic property of the ring R_q enables this compression? What speed gain does it provide?

5.4. Ask an LLM: *"In lattice-based cryptography, why is solving RLWE considered at least as hard as solving the closest vector problem (CVP)? Explain the reduction intuitively."* Verify that it mentions the role of the noise e in the connection to CVP.

5.5. Work in the ring $R_5 = \mathbb{Z}_5[x]/(x^4 + 1)$. Multiply $a(x) = 3x^3 + 2x^2 + 1$ by $b(x) = x^2 + 2x$, fold the terms of degree ≥ 4 using the rule $x^4 = -1$, and reduce the coefficients modulo 5. What is the resulting polynomial?

5.3 Packing and SIMD

We already know what we will be working with — polynomials of the ring R_q — but not what to put inside them. As we saw in Chapter 4, each homomorphic operation costs milliseconds regardless of how much data the

ciphertext contains, so packing as much as possible into each one is the key to better performance. Thanks to a technique called *batching* or *packing*, a single polynomial — and therefore a single ciphertext — can hold thousands of independent values, and a single operation on the ciphertext processes *all of them in parallel*. Packing is what turns FHE from a theoretical curiosity into a tool with acceptable performance — it's the reason Daniel's code processes Elena's thousands of salaries almost as fast as it would process just one. But how do thousands of numbers fit inside a single polynomial? The answer is the same idea we just saw with the NTT: representing a polynomial by its evaluations.

To understand why packing is necessary, consider a simple example: how could we encode the five salaries $[3200, 4100, 2800, 5500, 3900]$ into one or more polynomials? The most direct way would be to encode each value as a degree-zero polynomial, that is, a constant — $m(x) = 3200$, $m(x) = 4100$, etc. — and encrypt each one separately. The problem is that, as we'll see in the next section, each ciphertext is a pair of polynomials of degree $N-1 = 8191$, and therefore with $N = 8192$ integer coefficients modulo q; in our example, about 400 KB per ciphertext. Of those 8192 coefficients, only the constant term would carry useful information; the other 8191 would be zeros. For five salaries we'd need five independent ciphertexts — 2 MB total — and five separate operations for any computation. A polynomial with 8192 coefficients carrying a single number is an absurd waste, like chartering an 18-wheeler to deliver a letter.

The algebraic structure of the polynomials in the ring R_q offers something much better: packing the five values — or up to $N/2 = 4096$ — into a *single* polynomial, whose result not only takes up less space, but on top of that a single operation processes all the values at once. The data isn't placed in the polynomial's coefficients: it's encoded in its *evaluations* at specific points in the complex plane that are, in fact, the roots of $x^N + 1 = 0$, the ring's modulus.

From coefficients to slots. Recall that R_q, the ring we defined earlier, consists of the polynomials modulo $x^N + 1$ with integer coefficients modulo q, that is, polynomials of degree less than N. At first glance, a polynomial like that has N coefficients and that's it — there are no "independent positions" where data can be placed. But something interesting happens when we evaluate a polynomial $m(x)$ of degree less than N at specific points — an operation called *canonical embedding* in algebra.

Note that the polynomial $x^N + 1$ has N complex roots: $\omega_k = e^{i\pi(2k+1)/N}$ for $k = 0, 1, \ldots, N-1$. These are points equally distributed on the unit circle of the complex plane — exactly the same kinds of roots that appear in the

FFT. Evaluating $m(x)$ at those N roots produces N complex numbers. But since the coefficients of $m(x)$ are real, the evaluations at conjugate roots are also conjugates: $m(\bar{\omega}_k) = \overline{m(\omega_k)}$. The roots group into $N/2$ conjugate pairs, and each pair contributes a single independent complex value. The $N/2$ resulting complex values are the *slots*.

Each slot is a complex number, so in principle one could pack up to $N/2$ complex numbers, or up to N real numbers if the real and imaginary parts were used separately. Thanks to conjugate symmetry, any vector of $N/2$ complex numbers corresponds to a polynomial with N real coefficients, so the ring doesn't rule it out. Even so, libraries like TenSEAL only use the real part of each slot, because when two vectors are multiplied, the real and imaginary parts of both would get mixed, and the result would no longer be the element-wise product we are after. That's why, in our code, CKKS packs $N_{\text{slot}} = N/2$ real values, one per slot. With `poly_modulus_degree = 8192`, we have $N_{\text{slot}} = 4096$ slots available.

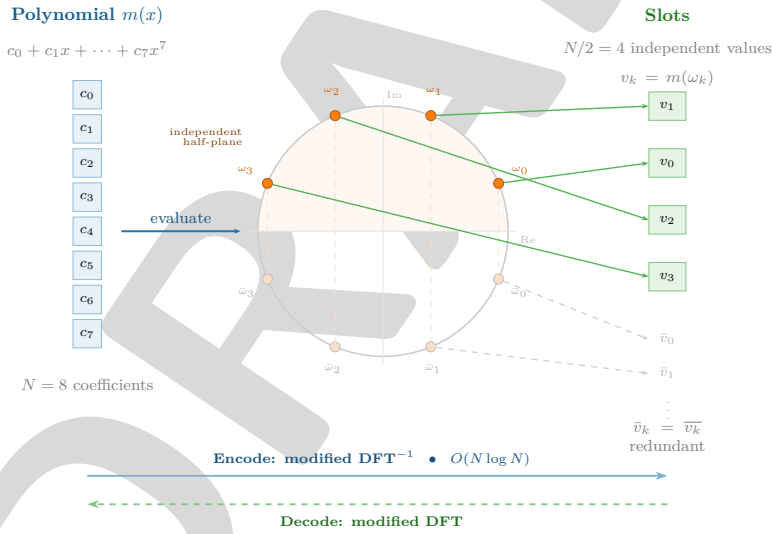


Figure 5.4: Canonical embedding. The polynomial $m(x)$'s coefficients are not the data: the slots are the evaluations of $m(x)$ at the N complex roots of $x^N + 1$. Conjugate roots produce redundant values, leaving $N/2$ independent slots. Encoding and decoding are equivalent to a modified DFT (inverse and direct, respectively).

Encoding a data vector — for example, the five salaries $[3200, 4100, 2800, 5500, 3900]$

— consists of finding the polynomial $m(x)$ of degree $N-1$ whose evaluations at the $N/2$ roots match those values. It's exactly an inverse Fourier transform: given the values in the "frequency domain" (the slots), we compute the signal in the "time domain" (the polynomial's coefficients). The library does this internally with a variant of the inverse DFT, which costs $O(N \log N)$ — the same complexity as the NTT we already use for polynomial multiplication. The five salaries are placed in the first 5 of the 4096 slots; the remaining 4091 are set to zero.

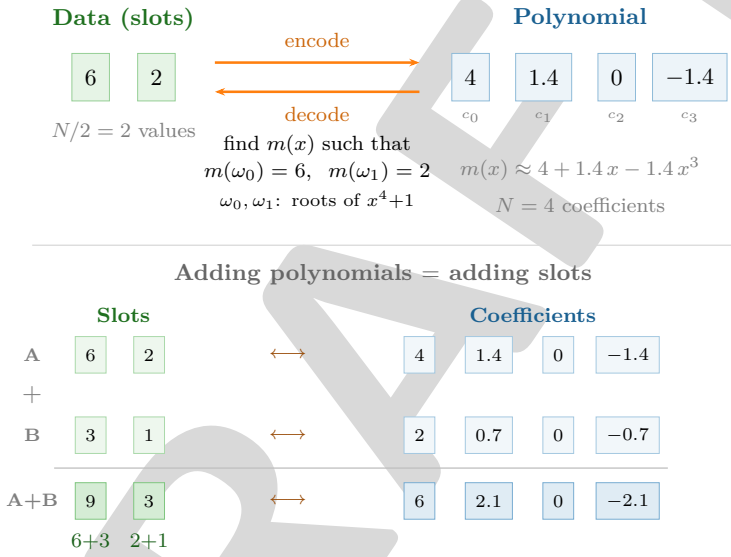


Figure 5.5: Slot-based encoding ($N = 4$). Above: two data points $[6, 2]$ are encoded as a polynomial with 4 coefficients by finding $m(x)$ such that $m(\omega_0) = 6$ and $m(\omega_1) = 2$. Below: adding two polynomials coefficient by coefficient is equivalent to adding the slots element by element — the basis of SIMD parallelism.

Scale and rounding. Let's go back to the example in the figure ($N = 4$, data $[6, 2]$). The polynomial whose evaluations at $\omega_0 = e^{i\pi/4}$ and $\omega_1 = \omega_0^3$ equal 6 and 2 is

$$m(x) = 4 + \sqrt{2}\,x - \sqrt{2}\,x^3.$$

This exposes a detail that the inverse transform doesn't solve on its own:

DRAFT

the coefficients are not integers, and R_q only admits integer coefficients.

> **i** Checking the $N = 4$ example
>
> In Cartesian form, $\omega_0 = \frac{\sqrt{2}}{2}(1+i)$ and $\omega_1 = \frac{\sqrt{2}}{2}(-1+i)$. When we subtract them, the imaginary parts cancel and the real parts add up, so $\omega_0 - \omega_1 = \sqrt{2}$. Since $\omega_1 = \omega_0^3$, the x^3 term of $m(x)$ equals $-\sqrt{2}\,\omega_1$ when evaluated at ω_0, and therefore
>
> $$m(\omega_0) = 4 + \sqrt{2}\,(\omega_0 - \omega_1) = 4 + \sqrt{2} \cdot \sqrt{2} = 6.$$
>
> With the other root, since $\omega_0^8 = 1$, we have $\omega_1^3 = \omega_0^9 = \omega_0$, and therefore $m(\omega_1) = 4 + \sqrt{2}\,(\omega_1 - \omega_0) = 4 - 2 = 2$.

To get integer coefficients, CKKS resorts to the solution used by any *fixed-point* system: multiply by a large scale factor Δ and round. With $\Delta = 1000$,

$$\lfloor \Delta \cdot m(x) \rceil = 4000 + 1414\,x - 1414\,x^3,$$

whose evaluations are 5999.7 and 2000.3, that is, the data multiplied by Δ with a small rounding error. That error is the first source of imprecision in CKKS, and Δ determines how many decimal places survive — in our code, `global_scale = 2**40`, about 12 digits. We'll hold on to one idea for later: every encoded polynomial carries its data multiplied by Δ. With the data now converted into a polynomial with integer coefficients, what remains is to see why operating on these polynomials amounts to operating on the data, and that is where the parallelism comes from.

Why does the parallelism work? This transformation doesn't just improve packing: it also parallelizes additions and multiplications on the ciphertext. The key is that operations in the ring R_q — addition and multiplication of the *plaintext* polynomials $m(x)$, before any encryption — automatically translate into element-wise operations on the slots. If $m_1(x)$ encodes the vector of evaluations $(v_1, v_2, \ldots, v_{N/2})$ and $m_2(x)$ encodes $(w_1, w_2, \ldots, w_{N/2})$, then:

- $m_1(x) + m_2(x)$ encodes $(v_1 + w_1,\ v_2 + w_2,\ \ldots,\ v_{N/2} + w_{N/2})$
- $m_1(x) \cdot m_2(x)$ encodes $(v_1 \cdot w_1,\ v_2 \cdot w_2,\ \ldots,\ v_{N/2} \cdot w_{N/2})$

Note that this product $m_1(x) \cdot m_2(x)$ is the ordinary multiplication of two *plaintext* polynomials within the ring. In the next section we'll see that operations on ciphertexts are built so that, when decrypted, they return

DRAFT

these sums and products of plaintext polynomials, up to a small error, and with them the slot-by-slot operations.

This is a direct consequence of the algebraic structure and of the transformation from values to slots, the polynomial's evaluations at the roots of $x^N + 1$: evaluating a polynomial at a point is a ring homomorphism — if $f(\omega) = v$ and $g(\omega) = w$, then $(f + g)(\omega) = v + w$ and $(f \cdot g)(\omega) = v \cdot w$. A single operation on the polynomial is equivalent to $N/2$ independent operations on the data — the same idea as *Single Instruction, Multiple Data* (SIMD) instructions in a modern processor.

In a CPU with AVX-512 instructions, a 512-bit register stores 8 64-bit *doubles*, and a single `VADDPD` instruction (*Vector Add Packed Double-Precision Floating-Point*) adds all 8 pairs. In FHE, the ciphertext plays the role of the "register" and stores up to $N_{\text{slot}} = N/2 = 4096$ values; a single homomorphic addition processes them all at once. The structure is the same; what changes is the scale of the cost: the AVX instruction takes on the order of a nanosecond, while the homomorphic operation takes on the order of milliseconds — the upper end of the 10^4–10^6 gap we saw in Chapter 4. What matters is that this cost is paid *per ciphertext*, not per value: it doesn't depend on how many slots are occupied, because *a single* polynomial operation is executed on the ring, whether it carries 5 values or 4096. Spread across the 4096 slots, the cost per value drops by about three orders of magnitude — and it's that amortization, not an intrinsically faster operation, that makes FHE viable.

Economics of packing. Without packing, each ciphertext stores a single value and occupies ~400 KB. Encrypting 4096 values would require 4096 ciphertexts — 1.6 GB — and 4096 operations to add them. With packing, the 4096 values fit in a single 400 KB ciphertext, and a single addition processes them all. The amortized cost drops from 400 KB to ~100 bytes per value, and the number of operations is reduced by a factor of $N/2$. It's this reduction that allows applications like encrypted regression (Recipe 3) or semantic search (Recipe 6) to be viable in practice.

In our code, when we write `ts.ckks_vector(contexto, salarios)`, the library internally executes that encoding: it transforms the Python vector `[3200, 4100, 2800, 5500, 3900]` into a polynomial of degree 8191 whose evaluations at the roots of $x^{8192} + 1$ reproduce those five values in the first slots. The programmer doesn't need to think about complex roots or transforms — just about vectors and arithmetic operations.

Now, packing has a limitation. It only allows *slot-by-slot* operations. Computing `salarios_cifrados.sum()` requires combining values that sit in

DRAFT

different slots, which can't be achieved with polynomial additions or multiplications; it needs a different mechanism, *rotations*, which we'll see in Section 5.5 as soon as we know how to encrypt. First, though, an even more basic piece is missing: encryption.

Review Questions

5.6. Why is the number of slots $N/2$ and not N? What property of the roots of $x^N + 1$ explains this?

5.7. In our example, `ts.ckks_vector(contexto, salarios)` places 5 values in 4096 slots. What do the remaining 4091 slots contain? Does it affect the cost of operations?

5.8. What happens to the data $[6, 2]$ from the example if it is encoded with $\Delta = 10$ instead of $\Delta = 1000$? Compute the rounded coefficients and their evaluations.

5.9. Imagine a database with a hundred thousand records. Given a polynomial degree of $N = 4096$ or 8192, is it possible to perform encrypted computations on vectors of a hundred thousand elements with TenSEAL or OpenFHE?

5.4 Encryption and homomorphic operations

We now have the algebraic structure — the polynomial ring R_q with its efficient operations via NTT —, we know that security resides in the difficulty of solving lattice problems, and we know how to turn a data vector into a polynomial with integer coefficients, $\lfloor \Delta \cdot m(x) \rceil$. In this section we'll call that already scaled and rounded polynomial the "message," so we don't have to drag Δ through every formula; in Section 5.6 we'll separate the scale from the data again to track what happens to Δ in each operation. Now let's see how a message is encrypted and, above all, why arithmetic operations on ciphertexts produce correct results without needing to decrypt — exactly what Daniel verified empirically in Recipe 1, when the encrypted average salary matched the one computed in plaintext.

The encryption mechanics are elegant. The secret key s is a polynomial with small coefficients. To encrypt a message m, a random polynomial a — unlike s and e, with coefficients uniformly distributed over the whole range modulo q, not small — and a small *error* e — noise deliberately injected to make the decryption problem harder — are generated, producing a pair (a, b):

$$\text{Enc}(m) = (a,\ b = a \cdot s + e + m)$$

Why is a necessary? Wouldn't it suffice to encrypt as $c = s + e + m$ and decrypt as $c - s$? The problem is that without the random polynomial a, if someone intercepts two ciphertexts $c_1 = s + e_1 + m_1$ and $c_2 = s + e_2 + m_2$, they can subtract them: $c_1 - c_2 \approx m_1 - m_2$. The key s cancels out and the attacker learns the difference between the messages. The random polynomial a, different in each encryption, acts as a mask that makes each ciphertext indistinguishable from random noise. Anyone who knows s can decrypt the ciphertext c — $\text{Dec}(c) = b - a \cdot s = e + m$ — and recover the plaintext message m (the error e is small and can be waved away as a precision error). Without knowing s, the presence of a and e makes recovering m equivalent to solving a lattice problem.

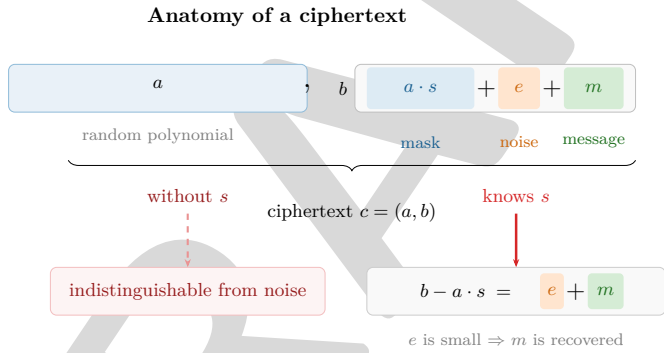


Figure 5.6: Anatomy of a ciphertext. The pair (a, b) contains the message m hidden behind a mask $a \cdot s$ and noise e. Anyone who knows s removes the mask and recovers m; without s, the ciphertext is indistinguishable from random noise.

The homomorphic property arises naturally. Let's see why. Let two ciphertexts $c_1 = (a_1, b_1)$ and $c_2 = (a_2, b_2)$ encrypt m_1 and m_2 respectively, where $b_i = a_i \cdot s + e_i + m_i$.

Addition. Simply add component by component:

$$\text{Enc}(m_1 + m_2) = c_1 + c_2 = (a_1 + a_2,\ b_1 + b_2)$$

If you decrypt this result with the key s, you get:

DRAFT

$$\begin{aligned}\text{Dec}(c_1 + c_2) &= (b_1 + b_2) - (a_1 + a_2) \cdot s \\ &= (e_1 + e_2) + (m_1 + m_2)\end{aligned}$$

That is, the decrypted message is m_1+m_2, the expected sum of the plaintext messages, with an error of $(e_1 + e_2)$. The error grows, but only as a sum — if each e_i is small, the sum remains small.

Addition with a plaintext constant. What happens if one of the operands is not encrypted? Let $c = (a, b)$ be a ciphertext encrypting m, and k a plaintext constant. The addition is even simpler:

$$\text{Enc}(m + k) = c + k = (a,\ b + k)$$

Only the second component is modified. When decrypting:

$$\text{Dec}(c + k) = (b + k) - a \cdot s = e + m + k$$

The error e doesn't change at all — the constant is added directly to the message without affecting the noise. This is the cheapest operation in the repertoire: there is zero error growth.

Multiplication by a constant. Now let's multiply the ciphertext $c = (a, b)$ by a plaintext constant k:

$$\text{Enc}(k \cdot m) = k \cdot c = (k \cdot a,\ k \cdot b)$$

When decrypting:

$$\begin{aligned}\text{Dec}(k \cdot c) &= k \cdot b - k \cdot a \cdot s = k \cdot (e + m) \\ &= k \cdot m + k \cdot e\end{aligned}$$

The message is $k \cdot m$ — correct — and the error goes from e to $k \cdot e$, proportional to the constant. The noise grows, but in a controlled way. Moreover, multiplying by a constant simply scales both components of the pair: the (a, b) structure is preserved intact, with no additional complications. There is one CKKS-specific wrinkle: if the constant is encoded with scale Δ, as TenSEAL does with any constant, the product ends up at scale Δ^2 and needs a *rescaling* that consumes a level, something we'll detail in Section 5.7. We'll see shortly that multiplication between *two ciphertexts*

DRAFT

is much more costly, since it requires extra machinery (relinearization) and makes the error grow much faster.

This explains why in the earlier recipes we insisted that operations like `features_cifrados * pesos` (Recipe 3) or `consulta_cifrada * scores` (Recipe 6) were "cheap" compared with multiplication between ciphertexts: when multiplying a ciphertext by plaintext values, no relinearization is needed and the error grows only in proportion to the constant, although, as we just saw, a level is consumed by the *rescaling*. Only multiplication between *two ciphertexts* — which we'll see next — adds relinearization as well as the contribution of the second ciphertext's error.

Multiplication between ciphertexts. Operations with plaintext constants were straightforward — component-wise additions and multiplications, with no extra cost. Multiplication between two ciphertexts isn't so simple.

We want to define $\text{Enc}(m_1 \cdot m_2) = c_1 \otimes c_2 = (a, b)$ — we use $\otimes$ instead of $\cdot$ because, as we'll see, this operation on ciphertexts is structurally different from constant multiplication — such that $\text{Dec}(c_1 \otimes c_2) = b - a \cdot s \approx m_1 \cdot m_2$. The question is: what must a and b be?

To find out, we work backwards. We know that decrypting each ciphertext gives $b_i - a_i \cdot s = m_i + e_i$, so the product of both decryptions is $(m_1 + e_1)(m_2 + e_2) \approx m_1 \cdot m_2$ — exactly what we want. We expand $(b_1 - a_1 s)(b_2 - a_2 s)$ to see what form it takes:

$$(b_1 - a_1 s)(b_2 - a_2 s) = \underbrace{b_1 b_2}_{d_0} - \underbrace{(a_1 b_2 + a_2 b_1)}_{d_1} \cdot s + \underbrace{a_1 a_2}_{d_2} \cdot s^2$$

The three coefficients — $d_0 = b_1 b_2$, $d_1 = a_1 b_2 + a_2 b_1$, and $d_2 = a_1 a_2$ — are computed exclusively from the public components, without needing s. So far so good.

But there's a problem: the expression doesn't have the form $b - a \cdot s$ that we need for decryption with a pair. It has *three* terms and depends on s^2. We can't find a pair (a, b) that fits directly — we need to eliminate that s^2.

This is where *relinearization* comes in. A *relinearization key* is published in advance: an encryption of s^2 under the key s itself, that is, a pair (a_r, b_r) where $b_r \approx a_r \cdot s + s^2$. With it, we absorb the $d_2 \cdot s^2$ term and construct the pair we were looking for:

$$a = d_1 + d_2 \cdot a_r, \qquad b = d_0 + d_2 \cdot b_r$$

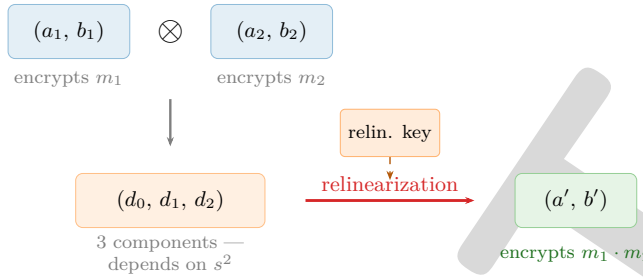


Figure 5.7: Multiplication between ciphertexts. The product generates a triplet (d_0, d_1, d_2) that depends on s^2. Relinearization, using a public key that encrypts s^2, reduces it to a standard pair (a', b') decryptable with s.

Does it work? Let's check: $b - a \cdot s = d_0 + d_2 b_r - (d_1 + d_2 a_r)s = d_0 - d_1 s + d_2(b_r - a_r s) \approx d_0 - d_1 s + d_2 s^2$, which is exactly the original product. Now we have a pair in the standard form that decrypts with just s and produces $m_1 \cdot m_2$.

This answers the question we posed: homomorphic multiplication is $\text{Enc}(m_1 \cdot m_2) = c_1 \otimes c_2 = (a, b)$ with a and b defined above. In libraries like TenSEAL, relinearization is applied internally every time you multiply two ciphertexts, so the user simply writes `c1 * c2`.

And since $\text{Dec}(c_1 \otimes c_2) \approx m_1(x) \cdot m_2(x)$, the product of the plaintext polynomials we saw in Section 5.3, homomorphic multiplication inherits the parallelism of packing: decrypting $c_1 \otimes c_2$ returns the slot-by-slot product $(v_1 w_1, \ldots, v_{N/2} w_{N/2})$, although with the data scaled by Δ^2 instead of Δ, something we'll correct in Section 5.6. Addition, by contrast, also operates slot by slot and keeps the scale Δ.

But once we can multiply, a second problem appears: the error grows much more rapidly. Notice that if we substitute $b_i = a_i s + e_i + m_i$, the decrypted product is:

$$(e_1 + m_1)(e_2 + m_2) = m_1 \cdot m_2 + e_1 \cdot m_2 + e_2 \cdot m_1 + e_1 \cdot e_2$$

The result contains $m_1 \cdot m_2$ — what we wanted — but the resulting error $e_{\text{mult}} = e_1 \cdot m_2 + e_2 \cdot m_1 + e_1 \cdot e_2$ is much larger than in addition. The errors no longer *add* but *multiply* with the messages, growing much faster. Notice that the first two terms, $e_1 \cdot m_2$ and $e_2 \cdot m_1$, are exactly the same growth we saw in multiplication by a constant — one ciphertext's error scaled by the

other's message — only now it happens in both directions at once, and the third term, $e_1 \cdot e_2$, is negligible next to the other two because both errors are small.

So, when messages and errors are of comparable magnitude, ciphertext multiplication isn't a distinct error-growth mechanism: it's roughly double a single constant multiplication, from the two symmetric contributions adding up. Still, that factor of 2 — constant and modest — isn't what should worry us: the real problem is that the error grows by multiplying with the messages at every level of the circuit, so after a few *chained* multiplications the accumulated error can exceed the message and decryption produces garbage. This is the fundamental reason why FHE has a limited operation budget.

What this noise budget is, and how many operations it allows before it runs out, we'll see in Section 5.6.

> **i** Only addition and multiplication
>
> The only two operations the scheme provides natively are polynomial addition and multiplication. The algebraic structure of RLWE — a polynomial ring — only defines these two operations. Any other computation — comparisons ($x > y$), maximums, minimums, non-linear functions like sigmoid or ReLU — must be *approximated* by a sufficiently high-degree polynomial, and each multiplication of that polynomial consumes a level from the budget. A simple `if (x > 0)` that in normal code is a single instruction requires evaluating a polynomial of degree 10 to 30 that approximates the sign function in FHE, consuming between 4 and 10 multiplicative levels. This restriction — sometimes called the *arithmetic circuit* model — is what turns programming on encrypted data into a careful design exercise, as we saw in Chapter 4.

Before that, however, one operation is still pending. Addition and multiplication only combine each slot with the one in the same position in the other ciphertext, but we already saw that `salarios_cifrados.sum()` needs to bring together values from different slots. That is what *rotations* solve, the third operation of the SIMD model.

DRAFT

Review Questions

5.10. In RLWE encryption, why is the random polynomial a necessary in each encryption? What attack does its absence enable?

5.11. Explain in your own words why homomorphic addition is "cheap" (the error grows linearly) while multiplication is "expensive" (the error grows much faster).

5.12. What role does *relinearization* play after a homomorphic multiplication? What would happen if it weren't applied?

5.13. CKKS doesn't support conditional branches. How would you implement the following logic using CKKS? Roughly how many multiplicative levels does it consume?

```
if (m < 0) then m = -m
```

5.5 Rotations

The ciphertext additions and multiplications we just saw operate element by element — slot by slot, as anticipated in Section 5.3 — and are the basis of SIMD parallelism in FHE. But many useful computations need to *combine* values that sit in different slots of the same ciphertext. Computing the mean of an encrypted vector requires summing all its elements; a matrix multiplication requires dot products between rows and columns; a convolution requires shifting and accumulating. All these operations require moving data between vector positions — exactly what `salarios_cifrados.sum()` does, the line Daniel wrote without a second thought in Recipe 1. Implemented naively, their cost would be proportional to the number of elements in the vector; *rotations* bring it down from linear to logarithmic.

Before seeing how rotations are implemented in the scheme — which involves automorphisms on polynomials and additional keys — it's worth understanding the idea at the vector level, without worrying yet about the underlying cryptographic machinery.

Cyclic shift. A rotation by k positions shifts all slots in the vector k positions to the left, cyclically, so that the value that was in slot i moves to slot $i - k$, and those that "fall off" one end reappear at the other. With four slots and a unit rotation as an example:

$$\mathrm{Rot}([a, b, c, d],\ 1) = [b, c, d, a]$$

The datum a from slot 0 moves to slot 3; the datum b from slot 1 moves to slot 0, and so on. It's a circular shift, not a loss of data — all values

DRAFT

remain in the ciphertext, just in different positions.

Summation via rotation tree. To sum all elements of an encrypted vector, there is no "sum all slots" operation — remember that the scheme only offers polynomial additions and multiplications. The solution is to rotate and add repeatedly. Let's illustrate with 4 slots:

$$[a,b,c,d] \xrightarrow{+\,\mathrm{Rot}(\cdot,2)} [a{+}c,\ b{+}d,\ c{+}a,\ d{+}b] \xrightarrow{+\,\mathrm{Rot}(\cdot,1)} [a{+}b{+}c{+}d,\ \ldots]$$

In the first step, we rotate by 2 positions and add the result to the original: each slot accumulates the sum of two elements. In the second step, we rotate by 1 position and add again: each slot contains the sum of all four. In general, $\log_2(N_{\text{slot}})$ steps suffice to sum the N_{slot} elements. With $N_{\text{slot}} = 4096$, that's 12 rotations and 12 additions — exactly what `.sum()` executes internally. The gain is substantial: without this trick, summing the values from N_{slot} slots would require $N_{\text{slot}} - 1$ individual operations — one accumulation per slot. The rotation tree reduces that cost from $O(N_{\text{slot}})$ to $O(\log_2 N_{\text{slot}})$: with 4096 slots, from 4095 operations to just 24. The idea is the same one that turns a linear search into a binary search — divide the problem in half at each step: each iteration doubles the number of elements accumulated in each slot, and after $\log_2 N_{\text{slot}}$ steps they all contain the total sum.

How does a rotation work on a polynomial? Here the mathematics of the ring R_q reveals an elegant trick. Recall that the slots are the evaluations of the polynomial $m(x)$ at the roots $\omega_0, \omega_1, \ldots, \omega_{N/2-1}$ of the polynomial $x^N + 1$. Permuting the slots — moving the value from slot i to slot j — is equivalent to permuting the roots at which the polynomial is evaluated.

It turns out that certain substitutions of the polynomial's variable — called *automorphisms* of the ring — do exactly that. The substitution $x \to x^k$ (for certain odd values of k) reorders the roots of $x^N + 1$ and therefore permutes the slots. Let's work through the example in the diagram (Figure 5.9): $N = 8$, 4 slots, substitution $x \to x^3$.

Step 1: what a slot is. In CKKS, slot j is the value the polynomial $p(x)$ takes when evaluated at the root $\omega^{5^j \bmod 16}$, where $\omega = e^{i\pi/8}$ is a primitive root of $x^8 + 1$. For $N = 8$ there are 4 slots, with roots $\omega^1, \omega^5, \omega^9, \omega^{13}$.

Step 2: what the substitution does to the polynomial. Applying $x \to x^3$ transforms $p(x)$ into the new polynomial $q(x) = p(x^3)$. The value of slot j in q is:

$$q(\omega^{5^j}) = p\Big((\omega^{5^j})^3\Big) = p\Big(\omega^{3\cdot 5^j \bmod 16}\Big)$$

DRAFT

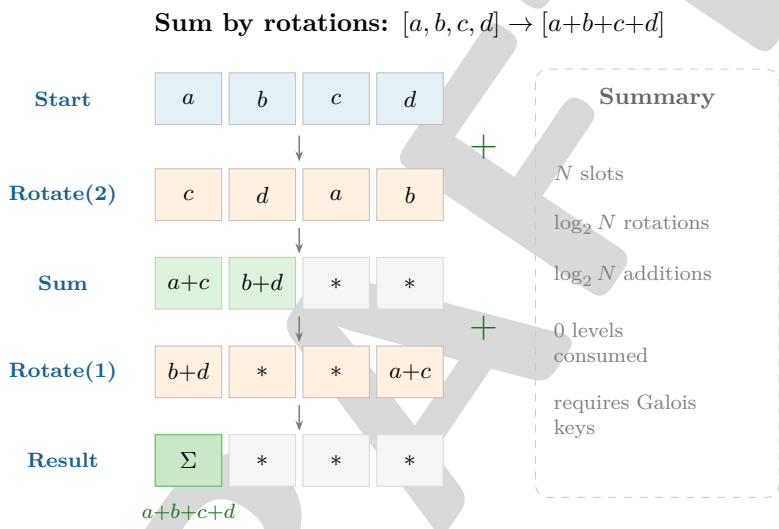


Figure 5.8: Summation via rotations. With 4 slots, two rotate-and-add steps suffice to accumulate the total sum in the first slot. The diagram shows the slot-level view; internally, each "Add" is an addition of the N coefficients of two polynomials, which by linearity of evaluation produces the slot-by-slot sum. Rotations require Galois keys but do not consume multiplicative levels (the budget of chained multiplications, which we will see in Section 5.6).

DRAFT

That is, slot j of the new polynomial takes the value p had at the root $\omega^{3 \cdot 5^j \bmod 16}$.

Step 3: working out where each slot goes. We compute $3 \cdot 5^j \bmod 16$ for $j = 0, 1, 2, 3$:

Slot j	Original root	$3 \cdot 5^j \bmod 16$	New root	Value in the slot
0	ω^1	$3 \cdot 1 = 3$	$\omega^3 = \overline{\omega^{13}}$	d (original slot 3)
1	ω^5	$3 \cdot 5 = 15$	$\omega^{15} = \overline{\omega^1}$	a (original slot 0)
2	ω^9	$3 \cdot 9 = 27 \equiv 11$	$\omega^{11} = \overline{\omega^5}$	b (original slot 1)
3	ω^{13}	$3 \cdot 13 = 39 \equiv 7$	$\omega^7 = \overline{\omega^9}$	c (original slot 2)

Step 4: why the conjugate gives the same value. In each case, the new root is the complex conjugate of another slot's root. For real data, $p(\bar{z}) = \overline{p(z)} = p(z)$ — the value at the conjugate matches the original value. Therefore slot j of the result contains the value of slot $j-1$ of the original.

The net effect is a **cyclic right rotation**: $[a, b, c, d] \to [d, a, b, c]$. Choosing different odd values of k yields different permutations — each k is a different rotation.

Why are Galois keys needed? Here's the practical problem. Applying $x \to x^k$ to the ciphertext $(a(x), b(x))$ produces $(a(x^k), b(x^k))$. This new pair still encrypts the rotated message, but under a *different* key: $s(x^k)$ instead of $s(x)$. It's as if rotating the glove box's gloves had changed the lock — the result is correct, but we can no longer open it with our original key.

To convert the ciphertext back to the key $s(x)$, an additional step called *key-switching* is needed — conceptually similar to the relinearization we saw in multiplication, but converting from $s(x^k)$ to $s(x)$ instead of from s^2 to s. This step requires a precomputed key that "translates" between the two keys. That key is the *Galois key* for rotation by k positions, and we need a different one for each value of k we want to use.

This explains why Galois keys are so voluminous. With $N = 8192$, the $\log_2(N/2) = 12$ rotations needed for a `sum()` use powers of 2 (1, 2, 4, …, 2048), and by default the library generates one key for each power of 2, both

DRAFT

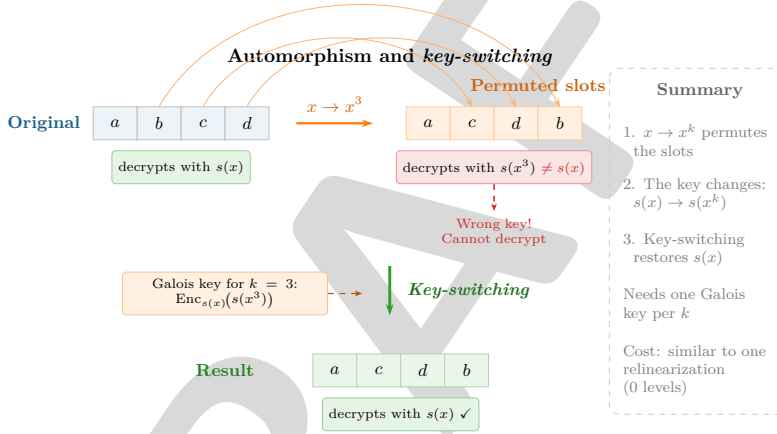


Figure 5.9: Automorphism and key-switching (example with $x^8 + 1 = 0$, $N = 8$, 4 slots). The substitution $x \to x^3$ cyclically permutes the slots: $[a, b, c, d] \to [d, a, b, c]$. But it changes the decryption key from $s(x)$ to $s(x^3)$. Key-switching, using a precomputed Galois key, restores the original key without revealing the secret.

DRAFT

positive and negative, plus one for conjugation, for 25 keys in total. Each one is a *key-switching* key containing several ciphertext-sized components, so it takes up considerably more space than a ciphertext (in TenSEAL, about 1.3 MB each, the same as the relinearization key, versus about 320 KB for a ciphertext), and the complete set comes to around 32 MB. A rotation by an amount that is not a power of 2 is decomposed into several rotations by powers of 2, so there's no need for a key for every possible position. With larger N, more levels, or additional rotations, the set can reach hundreds of megabytes.

Let's look at the table below to see how the keys we need have piled up:

Table 5.2: Keys in a CKKS context and their function.

Key	Generated	Enables	What happens without it?
Secret (s)	Automatically when creating the context	Decryption	Results cannot be recovered
Public	Automatically when creating the context	Encryption	Data cannot be encrypted (only the holder of s could)
Relinearization	Automatically in TenSEAL	Multiplication between ciphertexts	Ciphertext grows after each multiplication until unmanageable
Galois	Explicitly: `generate_galois_keys()`	Rotations	`sum()` and any operation combining slots fails with an error

In our code, `ts.context(...)` automatically generates the secret, public, and relinearization keys. Galois keys must be requested explicitly with `generate_galois_keys()`. Why aren't they generated automatically too? Because they're the bulkiest: with $N = 8192$, Galois keys take up about 32 MB, versus a little over 1 MB for the relinearization key, and their size grows quickly with N and with the number of levels. If the circuit only needs element-wise additions and multiplications — without rotations — that cost is unnecessary. So the library leaves the decision to the programmer.

If Daniel forgot that line and tried to run `salarios_cifrados.sum()` — which internally needs to rotate and accumulate the vector's slots — he'd get a runtime error. Element-wise additions and multiplications would work fine, but any operation that combines different slots of the vector would fail.

DRAFT

Cost of rotations. Rotations don't consume multiplicative levels — the budget of chained multiplications we will see in Section 5.6 — since they're linear operations, like additions. But they're not free: each rotation adds a small amount of noise (from the key-switching) and has a computational cost similar to a multiplication, because the key-switching step involves polynomial multiplications. In a circuit that combines packing with rotations, the designer must account for both the multiplicative depth and the number of rotations to correctly dimension the parameters.

Which operations benefit? The three primitives of the SIMD model — slot-by-slot addition, slot-by-slot multiplication, and rotation — cover a well-defined catalog of efficient operations. *Element-wise operations* (adding two encrypted vectors, multiplying them component-wise, scaling by a constant) are direct: a single polynomial operation across all slots in parallel, regardless of how many there are. *Reductions* — summing all elements, computing a mean, obtaining a norm — use the rotation tree at cost $O(\log N_{\text{slot}})$ instead of the $O(N_{\text{slot}})$ it would cost to operate slot by slot. *Dot products* combine both primitives: a SIMD multiplication (slot by slot) followed by a reduction via rotations. *Matrix-vector multiplications* decompose into dot products of each row with the vector. And neural network *convolutions* are implemented as sequences of rotations and accumulations. The general pattern is: any computation expressible as combinations of additions, multiplications, and shifts on vectors translates efficiently to FHE with packing and rotations. What falls outside are operations that depend on the *value* of the data — comparisons, conditionals, sorting — because the scheme doesn't allow inspecting a ciphertext to decide which branch to take.

Linear transformations. The tree summation is a special case of something more general: rotating, multiplying by plaintext constants, and adding are enough to apply *any* linear transformation to the slots, for example a matrix-vector product $y = Mv$ with a plaintext matrix M. Let's see it with two slots, $v = [a, b]$, and $M = \begin{pmatrix} p & q \\ r & s \end{pmatrix}$, whose expected result is $[pa + qb,\ ra + sb]$. Let's call *diagonal* 0 of M the vector $[p, s]$, and *diagonal* 1 the vector $[q, r]$, which we get by moving one column to the right in each row, wrapping around cyclically. If we multiply v, slot by slot, by diagonal 0 and add the rotated vector $\text{Rot}(v, 1) = [b, a]$ multiplied by diagonal 1, we get

$$[p, s] \cdot [a, b] + [q, r] \cdot [b, a] = [pa + qb,\ ra + sb],$$

where each product is slot by slot. That's exactly Mv, computed with just

DRAFT

one rotation, two multiplications by constants, and one addition. With $M = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ we get $[a+b,\ a-b]$, the two-point Fourier transform that will reappear in Section 5.8. With n slots we need n diagonals, that is, on the order of n rotations, which would be prohibitive with thousands of slots. That's why implementations don't apply the transform in one shot, but factor it, just as the FFT does, into a few very sparse layers, each with only a few diagonals. Each layer consumes one level for its multiplications by constants (a concept we will see in Section 5.7), and that is the tradeoff set by the `level_budget` in Recipe 5. More layers spend more levels, but each one needs fewer rotations and therefore fewer Galois keys. It's also how bootstrapping will later apply the packing transforms, *CoeffsToSlots* and *SlotsToCoeffs*.

In summary: packing reduces the *per-datum* cost when operations are parallel and independent, and rotations allow combining data from different slots when the circuit requires it. Together, they transform the economics of FHE — the amortized cost drops by several orders of magnitude and complex operations like means, dot products, and regressions become viable. The price is the additional complexity of Galois keys and the careful design of rotation patterns. But each rotation adds noise and each multiplication adds much more, so we still need to understand how many operations can be chained before the error destroys the message.

Review Questions

5.14. To sum all elements of an encrypted vector with $N_{\text{slot}} = 4096$ slots, how many rotations does the summation tree need? How many homomorphic additions?

5.15. Why does each rotation position need its own Galois key? What happens to the encryption key when applying the substitution $x \to x^k$?

5.16. Ask an LLM: *"What is the difference between slot rotation in FHE and a rotation (circular shift) in a CPU SIMD register? How are they similar and how do they differ?"*

5.6 Noise and levels

We've seen that every homomorphic operation makes the error grow — a little for a rotation, because of key-switching, and dramatically for a multiplication between ciphertexts, because errors multiply with the messages instead of simply adding up. After a few chained multiplications, the error

can exceed the message and decryption produces garbage. But exactly how many operations can we chain, and how does the scheme control that deterioration? It's exactly what Daniel wished he'd known before Elena called him one Friday at five in the afternoon to say the numbers were overflowing (Recipe 4).

The key is the modulus q, called the *ciphertext modulus*, since all the scheme's arithmetic operations are performed modulo q (in our code, $q \approx 2^{140}$: the 200 bits of `coeff_mod_bit_sizes = [60, 40, 40, 60]` minus the 60 of the last value, which is reserved for the keys and never becomes part of a ciphertext; we'll cover this in detail in Section 5.7). After encrypting a fresh message, the error e is small — on the order of a few bits. Decryption works correctly as long as the accumulated error is less than $q/2$. We can picture it as a fuel tank: $\log_2(q)$ — the number of bits of q — would be the size of the tank, and $\log_2(e)$ the fuel already consumed.

How much does the error grow in each multiplication? As we saw in Section 5.3, CKKS doesn't encode the data as is, but multiplied by a scale factor Δ and rounded (in our code, $\Delta = 2^{40}$). So decrypting a freshly encrypted ciphertext doesn't give us the value m, but $\Delta \cdot m + e$, where e bundles the rounding error of the encoding and the noise added during encryption. When multiplying two ciphertexts:

$$(\Delta\, m_1 + e_1)(\Delta\, m_2 + e_2) = \Delta^2\, m_1 m_2 + \Delta(m_1 e_2 + m_2 e_1) + e_1 e_2$$

The desired term is $\Delta^2 m_1 m_2$ and the error is $\Delta(m_1 e_2 + m_2 e_1) + e_1 e_2$. Since e_1, e_2 are small, the term $e_1 e_2$ is negligible and the dominant error is $\Delta(m_1 e_2 + m_2 e_1)$. As you can see, the multiplication has created two new problems: the message scale has gone from Δ to Δ^2, and the error has been multiplied by Δ.

Rescaling solves both in one stroke: we divide the entire ciphertext by Δ:

$$\frac{\Delta^2 m_1 m_2 + \Delta(m_1 e_2 + m_2 e_1)}{\Delta} = \Delta\, m_1 m_2 + (m_1 e_2 + m_2 e_1)$$

After rescaling, the message is back to being scaled by Δ — the standard form — and the error is now $e_{\text{resc}} = m_1 e_2 + m_2 e_1$. The extra 40 bits of error introduced by the multiplication have been eliminated by dividing by $\Delta = 2^{40}$. The resulting error is larger than the originals e_1, e_2 (by the factor m_i), but it no longer grows catastrophically. Without rescaling, each multiplication would add 40 bits of error and in two or three operations it

DRAFT

would overflow q; with rescaling, the error grows moderately and what gets consumed is a *link* of the modulus, an idea we'll make precise in Section 5.7.

But a ciphertext isn't a number; it's a pair of polynomials. How is this division by Δ carried out in practice? If we recall that in Section 5.3 the data wasn't in the coefficients but in the polynomial's evaluations, how do we know that dividing the polynomial doesn't ruin the packed message? And finally, why does that division change the ciphertext's modulus?

We'll answer in two steps. First we'll see how rescaling works step by step with a simple numerical example, treating each ciphertext as a single number. Then, in the next section, we'll redo the computation with actual polynomials.

Numerical example: rescaling step by step

Let's see this in simplified form with concrete numbers instead of polynomials — with $\Delta = 1000$ instead of 2^{40} the arithmetic can be followed by hand. Let $m_1 = 2.5$, $m_2 = 3.0$, with errors $e_1 = 2$ and $e_2 = -3$. Each fresh ciphertext internally stores $\Delta m + e$:

$$\Delta m_1 + e_1 = 1000 \times 2.5 + 2 = 2502, \qquad \Delta m_2 + e_2 = 1000 \times 3.0 - 3 = 2997$$

Multiplying the ciphertexts is equivalent to multiplying these two integers: $2502 \times 2997 = 7\,498\,494$. Compared to the "ideal" value $\Delta^2 m_1 m_2 = 1000^2 \times 2.5 \times 3.0 = 7\,500\,000$, the difference is -1506 — exactly $\Delta(m_1 e_2 + m_2 e_1) + e_1 e_2 = 1000 \times (-1.5) + (-6) = -1506$, the formula above confirmed with real numbers.
Now the rescaling: we divide by $\Delta = 1000$ and round to the nearest integer:

$$7\,498\,494 \;/\; 1000 = 7498.494 \;\approx\; 7498$$

That 7498 is the rescaled ciphertext, back to the standard scale $\Delta = 1000$ — the same magnitude as a freshly encrypted ciphertext, ready for another multiplication. Decrypting, $7498/1000 = 7.498$, practically the exact value $m_1 m_2 = 7.5$. The small discrepancy (0.002) comes from two sources: the genuine noise $(m_1 e_2 + m_2 e_1)/\Delta = -0.0015$ we already saw in the formula, and the rounding from dividing by Δ, which contributes at most half an integer unit — here, 0.494, or 0.000494 after decoding. Both sources are tiny compared to the message itself; it's precisely that margin that rescaling protects on every multiplication.

DRAFT

Contrast this with multiplying by a plaintext integer constant, say $k = 4$, instead of by m_2. The ciphertext is still $\Delta m_1 + e_1 = 2502$; multiplying it by k gives $k(\Delta m_1 + e_1) = 4 \times 2502 = 10008$. Compared to the ideal value $\Delta(km_1) = 1000 \times 4 \times 2.5 = 10000$, the difference is 8 — exactly $ke_1 = 4 \times 2 = 8$, the same noise scaled by k we saw in "Multiplication by a constant." The key difference is in the scale: 10008 is already at the standard scale $\Delta = 1000$ (decrypting, $10008/1000 = 10.008 \approx km_1 = 10$), because k is a bare number without its own Δ factor — unlike m_2, which did carry one inside the second ciphertext. There's no Δ^2 to correct and therefore, in principle, nothing to rescale if k is applied as is, without encoding it with Δ, since multiplying by a small *integer* only grows the noise by the factor k. We'll see in the next section that the situation changes when the constant is fractional, like $1/n$, or a vector of weights, because it has to be encoded with scale Δ; and that TenSEAL encodes all constants that way, integers included.

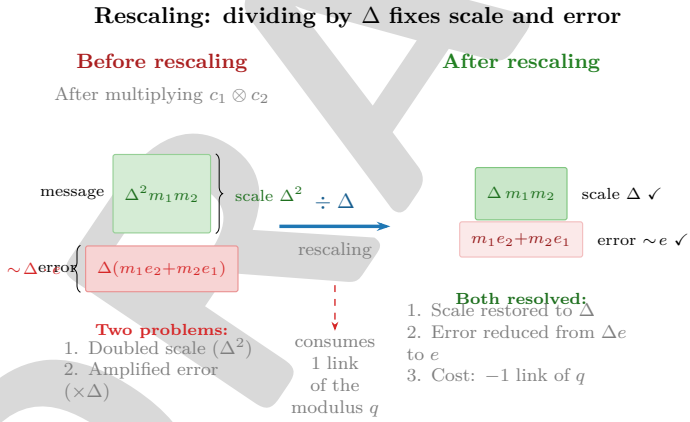


Figure 5.10: Rescaling: dividing by Δ simultaneously corrects the doubled scale and the amplified error, at the cost of consuming one link of the modulus chain.

The example has only half-answered our questions: we've seen *what* rescaling does, but with bare numbers, no modulus, and no packing. It's time to do the same computation with polynomials.

DRAFT

5.7 Polynomial division and rescaling

Let's now repeat the previous numerical example, but with polynomials, and answer the same three questions: how is the division carried out? How do we know packing doesn't get ruined? Why does the modulus change?

The first is the easiest. "Dividing a polynomial by Δ" is nothing exotic: it means dividing each of its coefficients by Δ and rounding to the nearest integer. If $c(x) = c_0 + c_1 x + \cdots + c_{N-1} x^{N-1}$, rescaling produces

$$\lfloor c(x)/\Delta \rceil = \lfloor c_0/\Delta \rceil + \lfloor c_1/\Delta \rceil \, x + \cdots + \lfloor c_{N-1}/\Delta \rceil \, x^{N-1}.$$

That's N independent integer divisions, exactly what we did with the 7 498 494 in the example, repeated for every coefficient.

Why doesn't it ruin the slots? Because evaluating a polynomial is a *linear* operation, so if we divide all the coefficients by Δ, the polynomial's value at any point also gets divided by Δ,

$$\frac{c(\omega)}{\Delta} = \frac{c_0}{\Delta} + \frac{c_1}{\Delta}\,\omega + \cdots + \frac{c_{N-1}}{\Delta}\,\omega^{N-1}.$$

And the slots are nothing more than those evaluations at the roots ω_k. Dividing coefficient by coefficient is therefore equivalent to dividing *every slot* by Δ, all at once. It's the same property that in Section 5.3 turned polynomial sums into slot-by-slot sums, now applied to a division by a number.

Let's see it with the two vectors of Figure 5.5 in Section 5.3 ($N = 4$, $\Delta = 1000$). We encode $[6, 2]$ as $4000 + 1414x - 1414x^3$ and $[3, 1]$ as $2000 + 707x - 707x^3$. Their product in the ring, folding with $x^4 = -1$, is

$$9\,999\,396 + 5\,656\,000\,x - 5\,656\,000\,x^3,$$

whose slots are 17 998 188 and 2 000 604, that is, the products $6 \cdot 3 = 18$ and $2 \cdot 1 = 2$ scaled by $\Delta^2 = 10^6$. Dividing each coefficient by 1000 and rounding gives $9999 + 5656\,x - 5656\,x^3$, with slots 17 997.8 and 2000.2. Undoing the scale Δ when decoding, we get [17.998, 2.0002]. No slot got mixed with another; all that appeared was the small rounding error from the encoding and from the rescaling itself.

And what about the ciphertext? So far we've divided the plaintext polynomial, but the server doesn't have it; it only has the pair (a, b) and

DRAFT

doesn't know s. What it does is divide and round *both components*, $a' = \lfloor a/\Delta \rceil$ and $b' = \lfloor b/\Delta \rceil$. Since the decryption $b - a \cdot s$ is linear, $b' - a' \cdot s$ is approximately $(b - a \cdot s)/\Delta$, that is, the message divided by Δ plus a small rounding error, because s has small coefficients. The server rescales without seeing anything.

Why does the modulus change? Here's the detail the numerical example hid, because there we worked with plain integers, with no modulus. In the real scheme, the coefficients of a and b are stored reduced modulo q, that is, as remainders between 0 and $q - 1$. Modular arithmetic works like a clock with q hours, where going past $q - 1$ starts over from zero, and reducing modulo q is equivalent to stripping out the full laps a number has made. That's why, when decrypting, the integer $b - a \cdot s$ isn't directly the message, but the message plus a whole number k of laps,

$$b - a \cdot s = \Delta^2 m_1 m_2 + e + k \cdot q,$$

where e is the error of the product, and it's the final reduction modulo q that eliminates the term $k \cdot q$ and leaves the message.

The server can't do that reduction before rescaling, because to compute $b - a \cdot s$ it would need s. What it does instead is divide a and b by Δ, as we just saw, and by linearity the division reaches all three terms at once,

$$b' - a' \cdot s \approx \Delta\, m_1 m_2 + \frac{e}{\Delta} + k \cdot \frac{q}{\Delta}.$$

The first two terms are the message back at scale Δ, with a smaller error, which is what we were after. But the laps have been divided too, and now they measure q/Δ instead of q. If we kept reducing modulo q when decrypting, the term $k \cdot q/\Delta$ wouldn't disappear and would contaminate the result. To erase it we have to reduce modulo q/Δ, that is, use a clock Δ times smaller.

With the numbers from the example, suppose $q = 10^{10}$, plenty of room for the 7 498 494, and that the integer $b - a \cdot s$ is 30 007 498 494, that is, three laps of 10^{10} plus the 7 498 494 we're after. Dividing by 1000 and rounding gives 30 007 498. Reduced modulo $q/1000 = 10^7$, we're left with 7498, the correct result of the example. Reduced modulo 10^{10}, on the other hand, it would still be 30 007 498, because the three divided laps (30 000 000) no longer make a full lap of that clock. That's why dividing the ciphertext by Δ *is*, at the same time, lowering its modulus from q to q/Δ; they aren't two operations that happen to coincide.

DRAFT

That same computation requires q/Δ to be an integer, or the laps would leave remainders that would contaminate the message. That's why q isn't an arbitrary number but a *product of primes* chosen in advance, and each rescaling doesn't divide by exactly 2^{40} but by one of those primes, $p \approx 2^{40}$. The resulting scale, Δ^2/p, is almost — though not exactly — Δ.

Repeated after each multiplication, this division by one of the primes produces a sequence of ever-smaller moduli, $q_0 > q_1 > q_2 > \cdots$, known as the *modulus chain*. Each step of the chain is what we've been calling "consuming a link" or a *level*, and since the chain is finite, so is the number of chained multiplications. Hence the name of *leveled* schemes, which fix the chain in advance and, with it, how many multiplications fit.

Plaintext constants. This mechanism also explains a subtlety of the recipes. In theory, multiplying by a small integer, like the $k = 4$ in the example, doesn't add any Δ. But a fractional constant, like $1/n$ or a vector of weights, has to be encoded like any other piece of data — with coefficients scaled by Δ — and the product ends up at scale Δ^2 again, so it needs its own rescaling and consumes a level, just like a multiplication between ciphertexts. TenSEAL, moreover, applies that same encoding to any constant, integers included, so even `ct * 4` consumes a level. That's what happened to Daniel with the `* (1/n)` in the variance in Recipe 2. The difference from a multiplication between ciphertexts isn't in the levels but in the cost: there's no relinearization, and the error grows only in proportion to the constant, without the contribution of the second ciphertext.

Now we can read our code's parameters with different eyes. The chain is exactly what `coeff_mod_bit_sizes` configures in TenSEAL:

```
coeff_mod_bit_sizes = [60, 40, 40, 60]
```

Each number is the size in bits of one of the primes:

- The two **intermediate 40s** are the primes we divide by at each rescaling — the links of the chain. Their size matches `global_scale = 2**40`, as the division by $p \approx \Delta$ we just saw requires. Two links, two chained multiplications.
- The **first 60** is the base prime, the only one that is never divided out. When the links run out, the ciphertext lives only modulo that prime, and its size, compared to Δ, sets the margin left for the integer part of the result, $60 - 40 = 20$ bits, a bit over a million.
- The **last 60** is the auxiliary modulus for the relinearization we saw earlier, the step that converts the triplet (d_0, d_1, d_2) into a pair. It only exists at the key level and never becomes part of a data ciphertext.

DRAFT

A freshly encrypted ciphertext therefore starts with $q_0 = 2^{60} \cdot 2^{40} \cdot 2^{40} = 2^{140}$ (approximately, because they're primes and not powers of two). Each multiplication followed by a rescaling divides it by one of the 40-bit primes, so $q_0 \approx 2^{140} \to q_1 \approx 2^{100} \to q_2 \approx 2^{60}$. In general, after ℓ multiplications the ciphertext lives modulo $q_\ell \approx q_0/\Delta^\ell$, and with L links the chain ends at q_L, the base prime (Figure 5.11).

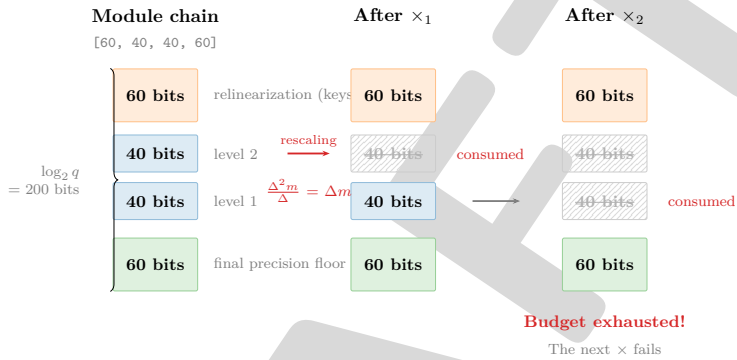


Figure 5.11: Modulus chain and level consumption. Left: the initial chain `[60, 40, 40, 60]` with its four values — the top 60-bit one is the auxiliary relinearization modulus, which only exists at the key level and never belongs to the data ciphertext; the two 40-bit values are the available multiplication links; the bottom 60-bit one is the base modulus that survives to the end, setting the precision floor. Center: after the first multiplication, rescaling divides by Δ and consumes the "level 2" link, leaving one available. Right: after the second multiplication, "level 1" is also consumed and the budget is exhausted — a third multiplication would have no link left to consume and would fail.

With this picture, the two failures in the recipes are two sides of the same coin. In Recipe 4, with `[60, 40, 60]`, the second multiplication produces a result at scale $\Delta^2 = 2^{80}$ that no longer fits in the 60 bits of the base prime, and there's no 40-bit prime left to divide by. TenSEAL detects this and raises "scale out of bounds." In the naive version of Recipe 2, on the other hand, the computation arrived right at q_L with a value larger than the 20 bits of margin of the base prime; the result made more laps around the clock than that modulus could tell apart, and nothing in the scheme can detect it — hence the negative variance with no error. In the first case a link is missing; in the second, margin is missing at the last one.

Why not use a larger Δ, if it gives more precision? Because Δ is also the

DRAFT

size of each link. A 40-bit Δ gives about 12 decimal digits (comparable to a *double*), but each multiplication spends 40 bits of the modulus. What has to fit in q_0 is not Δ but Δ^L — the L links — plus the base prime. If Δ were comparable to q_0, there would be room for a single rescaling, and it would be like building a wall with a single brick the size of the entire wall. That's why moderate values are used, typically between 30 and 60 bits.

How does an engineer decide these values? The rule of thumb is: count how many chained multiplications your circuit needs (call it L) and configure L intermediate values of 40 bits (for about 12 decimal digits of precision), plus a 60 at the beginning and another at the end. To compute a mean (sum + division by constant) a single multiplication suffices, so $L = 1$ would be enough. Our example uses $L = 2$ for headroom. If you needed, for example, to compute a degree-4 polynomial, you'd use `[60, 40, 40, 40, 40, 60]` — four intermediate levels. More levels imply a larger total modulus q, which in turn requires a larger `poly_modulus_degree` to maintain security.

Why is a given value chosen for q, the total modulus, Δ, the rescaling factor, and N, the lattice dimension? Recall that RLWE security rests on the fact that the error e injected during encryption must be small enough to be removed during decryption, but large enough to "hide" the message from an attacker. An attacker trying to break the scheme must distinguish an RLWE ciphertext — $b = a \cdot s + e + m$ — from a pair of purely random polynomials.

The larger q is relative to the error e, the easier it becomes to distinguish the ciphertext from pure noise: the error occupies a smaller fraction of the space of possible values modulo q, and the ciphertext increasingly resembles an exact product $a \cdot s$ without noise, which would be trivial to invert. Increasing N — the dimension of the underlying lattice — compensates for that weakening, because it makes the problem exponentially harder for the same reason we saw when introducing RLWE: the best lattice reduction algorithms have exponential cost in the dimension.

As a rule of thumb, the *security bits* — the exponent λ such that an attacker needs $\approx 2^\lambda$ operations to break the scheme by brute force — are approximately proportional to $N/\log_2 q$. This has a direct consequence: if you double the size of q by adding levels to the chain, you need to increase N in the same proportion for the attack cost to remain exponential.

For example, with 4 levels the chain `[60, 40, 40, 40, 40, 60]` totals 280 bits of modulus, versus the 200 bits in our original example (counting the auxiliary modulus in both cases). To maintain 128 bits of security with that larger modulus, you'll typically need $N = 16384$ instead of 8192.

DRAFT

i The trade-off of N: security, parallelism, and cost

We've just seen that N isn't a free choice: the size of the modulus q imposes a minimum N to preserve security. But `poly_modulus_degree`, the parameter that sets N in the code in Chapter 2, also decides two other things. On one hand, parallelism, since each ciphertext can hold $N_{\text{slot}} = N/2$ values, as we saw in Section 5.3. On the other, cost, since every operation scales as $O(N \log N)$, and N can't grow gradually because, as we saw when introducing the NTT, it must be a power of 2 (4096, 8192, 16384…). Doubling N from 8192 to 16384 doubles N_{slot}, but it also (roughly) doubles the time of every addition or multiplication, and the size of every ciphertext — which, as we saw in Chapter 4, is already around 400 KB with $N = 8192$.

But if the budget is finite, something doesn't add up: we call the scheme *fully* homomorphic, and yet it only allows a fixed number of multiplications — not very different, in principle, from the partial schemes we mentioned at the beginning of the chapter.

Review Questions

5.17. Why is *rescaling* necessary after each multiplication in CKKS? What happens to the message scale if it's not applied?

5.18. For a circuit that computes $(a \cdot b) \cdot c$, how many levels are needed? Compare it with $a \cdot b + c \cdot d$. What determines the number of levels required: the total number of multiplications or the depth of the circuit?

5.19. Why does dividing all the coefficients of a polynomial by Δ also divide every slot by Δ, without mixing slots with one another?

5.20. With `coeff_mod_bit_sizes = [60, 30, 30, 30, 60]` and `global_scale = 2**30`, how many bits does the modulus of a freshly encrypted ciphertext have, and after each multiplication? How much margin is left for the integer part at the end? What do you gain and what do you lose compared to the chapter's configuration?

5.8 Bootstrapping

We've traveled a path from lattices to algebra and from there to polynomials in RLWE; from data packed into slots to encryption and homomorphic

operations, including the rotations that move data between slots; and from noise growth to rescaling, which keeps it in check by dividing the ciphertext and stepping down one rung of the modulus chain. But everything described so far defines a *leveled* scheme — capable of a prefixed number of multiplications. To earn the name *fully* homomorphic (the F in the FHE acronym), we need a way to restore the exhausted budget. That is the key contribution of Craig Gentry in 2009, which we already mentioned in the history of FHE, and it is called *bootstrapping*; it is the same technique Daniel applied in Recipe 5 to rescue the computation that had left Elena staring at the "spinning wheel of death" on her screen.

Suppose that after L multiplications we've exhausted the modulus chain: each *rescaling* has divided the modulus by Δ, leaving the ciphertext at the last link $q_L = q_0/\Delta^L$, with no levels left to keep operating. The error is small thanks to *rescaling*, but there are no more links to consume.

The bootstrapping idea is counterintuitive: we take that "exhausted" ciphertext and decrypt it *homomorphically*. That is, the decryption circuit itself $(b - a \cdot s)$ is evaluated as an operation on ciphertexts. To do this, the secret key s is encrypted in advance under the key s itself and published as the *bootstrapping key* (BK). Returning to Gentry's analogy from the beginning of the chapter, the system builds a new "glove box" inside the exhausted glove box, replacing it: the exhausted ciphertext goes in, gets decrypted internally and re-encrypted, all without ever leaving the encrypted domain. The result is a *fresh* ciphertext — with the modulus restored to q_0 and the full level budget — containing the same message.

Let's see step by step how it works. We start from the exhausted ciphertext $c = (a, b)$ at the last level L, with modulus q_L. We know that:

$$b - a \cdot s \equiv \Delta\, m + e \pmod{q_L}$$

where e is small thanks to accumulated rescaling. The values a and b are known — they are the ciphertext itself, which the server has in its hands. What it doesn't know is s.

Before starting any computation, the key owner has published a *bootstrapping key* (BK): a fresh encryption of the secret key s under the key s itself, but with the full modulus q_0. Following the same encryption notation we defined earlier — $\mathrm{Enc}(m) = (a,\ b = a \cdot s + e + m)$ — the bootstrapping key is:

$$\mathrm{BK} = \mathrm{Enc}(s) = (a_{\mathrm{bk}},\ b_{\mathrm{bk}} = a_{\mathrm{bk}} \cdot s + e_{\mathrm{bk}} + s)$$

DRAFT

It's exactly a normal encryption, except the "message" being encrypted is the key s itself. (Encrypting the key under itself requires an assumption called *circular security*; all current FHE schemes rely on it.) The crucial difference is that this encryption was generated with the full modulus q_0, so it has its entire level budget intact.

Now the server can evaluate the decryption circuit homomorphically. Let's see how, step by step. Recall that a and b — the components of the exhausted ciphertext — are concrete values, polynomials that the server has in its hands. All the information about the message m and the accumulated error e is encoded *within* those values, in the relationship $b - a \cdot s \equiv \Delta m + e \pmod{q_L}$. The polynomials a and b themselves are just big numbers; the error doesn't "live" in a or b separately, but in how they combine with the key s.

But there's a crucial detail in that " $\pmod{q_L}$". What it means is that the *integer* value of $b - a \cdot s$ — without reducing modulo anything — is not $\Delta m + e$, but:

$$b - a \cdot s = \Delta\, m + e + k \cdot q_L$$

for some integer k. When reducing modulo q_L, the term $k \cdot q_L$ disappears and only the residue $\Delta m + e$ remains. When the ciphertext operated under modulus q_L, that didn't matter — the reduction was applied implicitly. But now that we're going to switch to a larger modulus, that $k \cdot q_L$ becomes a problem. Let's see it step by step:

Step 1: Homomorphic decryption. The server uses a and b as known constants to operate on the BK. First, it multiplies the BK (which is a ciphertext containing s) by the constant a. We already saw that multiplying a ciphertext by a constant preserves the message: if BK encrypts s, then $a \cdot \text{BK}$ encrypts $a \cdot s$. This operation doesn't need relinearization — it isn't a multiplication between two ciphertexts — and it doesn't consume a level either, unlike the constants encoded with scale Δ that we saw in Section 5.7. Here a is not a piece of data encoded with scale Δ, but a whole polynomial that is multiplied as is, so no Δ^2 appears that would need rescaling. It does spike the noise, though: as we saw with multiplication by a constant, the error scales with the multiplier itself, and here the multiplier isn't a small constant but a, a polynomial with coefficients as large as the modulus q_L itself. The BK's noise gets multiplied by something on the order of q_L, not by a modest factor — which is precisely why the BK is encrypted under the full modulus q_0: it needs that extra headroom to absorb a noise jump of that magnitude without overflowing. Then it

DRAFT

negates the result and adds the constant b — negating is multiplying by the constant -1, and adding a constant to a ciphertext simply adds that constant to the encrypted message; neither operation alters the noise. We write that combination compactly as $b - a \cdot \mathrm{BK}$, where a and b are the known components of the exhausted ciphertext (not to be confused with $a_{\mathrm{bk}}, b_{\mathrm{bk}}$, the internal components of BK). Since Enc always produces a pair, that compact notation abbreviates the actual pair, obtained by negating and adding component by component:

$$b - a \cdot \mathrm{BK} = (-a \cdot a_{\mathrm{bk}},\ \ b - a \cdot b_{\mathrm{bk}}) = \mathrm{Enc}(b - a \cdot s) = \mathrm{Enc}(\Delta\, m + e + k \cdot q_L)$$

This new ciphertext was generated from the BK, which has the full modulus q_0. So it has plenty of levels to keep operating. But it contains $\Delta m + e + k \cdot q_L$ — the message we want *plus* a spurious term $k \cdot q_L$ that must be eliminated.

Step 2: Elimination of the $k \cdot q_L$ term (homomorphic modular reduction). This is where the real cost of bootstrapping lies. We need to compute $(b - a \cdot s) \bmod q_L$ homomorphically to keep only $\Delta m + e$. But modular reduction is not a polynomial operation — it cannot be expressed as a sequence of additions and multiplications. To evaluate it homomorphically, it must be *approximated* by a high-degree polynomial, which consumes multiple multiplication levels — typically between 10 and 20, depending on the desired precision.

Only after this second step do we obtain a fresh ciphertext under q_0 that contains $\Delta m + e$ — the clean message, without the $k \cdot q_L$ term, and with the level budget partially restored.

That's why bootstrapping is expensive: Step 1 ($b - a \cdot \mathrm{BK}$) is trivial, but Step 2 (the modular reduction) is a heavy computation that consumes a good portion of the newly restored levels.

It's like draining an airplane's fuel tank down to the reserve and, instead of landing, using that very reserve to power a pump that refills the main tank from an auxiliary one you'd been carrying on board.

In practice, a bootstrap takes on the order of one second — compared to the milliseconds of a normal operation — and the resulting ciphertext doesn't recover *all* the levels, because the decryption circuit evaluation itself consumes part of the restored budget. For this reason, the preferred approach is to design the computation to fit within the level budget (*leveled FHE*) and resort to bootstrapping only when the circuit is too deep to dimension in advance.

DRAFT

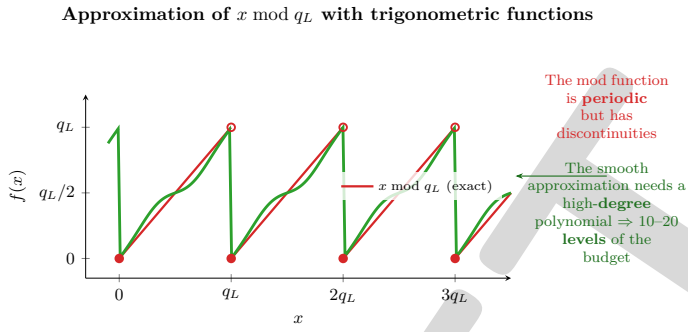


Figure 5.12: Approximation of modular reduction. The function "x modulo q_L" is periodic but discontinuous — it cannot be expressed as a sequence of additions and multiplications. To evaluate it homomorphically, it is approximated by a high-degree polynomial, which consumes between 10 and 20 multiplicative levels.

A CKKS detail: coefficients versus slots. Everything above assumes we can operate on the polynomial's coefficients, which is where the spurious term $k \cdot q_L$ shows up. But in CKKS the data is not stored as coefficients; it is encoded with the transform we saw in Section 5.3, so each *slot* is an evaluation of the polynomial and mixes all of its coefficients. That clashes with Step 2, because the reduction modulo q_L has to be applied to each coefficient separately, while homomorphic operations, including the polynomial approximation of that step, act slot by slot. Applied to the slots, it would make no sense, because each one is a combination of many coefficients.

The solution is to add two steps to the bootstrapping circuit. *CoeffsToSlots* converts the ciphertext into another one whose slots contain the polynomial's coefficients, so that the reduction in Step 2 acts precisely on what it should. *SlotsToCoeffs* undoes the change at the end and returns the data to its usual encoding. They are, in essence, the packing transform and its inverse, now applied to a ciphertext.

How can a transform be applied without decrypting? Because it is a *linear* operation on the slots whose constants are public — they depend only on N, not on any data — and we saw in Section 5.5 that an operation like that is built from nothing but rotations, multiplications by plaintext constants, and additions. The smallest example is the two-point Fourier transform, which turns the slots $[a, b]$ into $[a+b,\ a-b]$ and is the basic building block

DRAFT

of the big ones. With the rotation $[b, a]$ and slot-wise products,

$$[1, -1] \cdot [a, b] + [1, 1] \cdot [b, a] = [a + b,\ a - b].$$

The vectors $[1, -1]$ and $[1, 1]$ are plaintext constants, so the whole computation is one rotation, two multiplications by constants, and one addition, all of them operations on the ciphertext. Real transforms are the same thing with many more slots, spread across several layers. At no point is anything decrypted or the secret key involved.

The price is levels and keys. Multiplications by constants consume levels (Section 5.7) and rotations need Galois keys, as we saw, which explains why bootstrapping spends levels of its own. OpenFHE lets you split each transform into several layers, and that is what the `level_budget = [2, 2]` in Recipe 5 sets. With more layers, more levels are spent, but each layer needs fewer rotations and bootstrapping ends up faster.

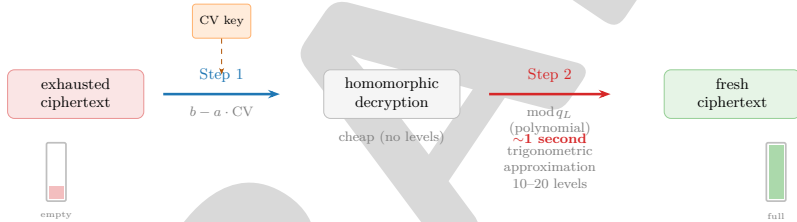


Figure 5.13: Bootstrapping in two steps. Step 1 is cheap: it multiplies the BK key by known constants. Step 2 is the bottleneck (approximately 1 second), because it approximates the modular reduction via a high-degree polynomial.

With or without bootstrapping, each homomorphic operation takes milliseconds, a cost that, thanks to packing, is spread across thousands of slots. With that, the machinery of CKKS — the scheme we use in our code — is complete: lattices, RLWE, packing, encryption and homomorphic operations, rotations, noise, rescaling, and bootstrapping. But this is not the only way to build an FHE scheme.

DRAFT

Review Questions

5.21. Why does bootstrapping need to encrypt the secret key under itself? What security assumption does this require?

5.22. If bootstrapping is so expensive, why aren't the parameters always dimensioned to avoid it? When is it unavoidable?

5.23. In CKKS, bootstrapping must apply the modular reduction to the polynomial's coefficients, but homomorphic operations act slot by slot. What is the *CoeffsToSlots* step for? What does it cost?

5.9 Other encryption schemes

Everything we've seen describes the CKKS scheme, but it's not the only FHE scheme. There are four main schemes, and choosing among them is an engineering decision that depends on the type of data, the operations needed, and the acceptable trade-offs.

The RLWE family: BFV, BGV, and CKKS. All three share the same foundations: the RLWE ring, the modulus chain, SIMD packing, and rotations via automorphisms. What differs is how they encode the message before encrypting it.

BFV and BGV work with *exact integers*: decryption recovers the value with no error at all, through exact rounding (BFV) or by reducing the message modulus (BGV). That makes them ideal for counting, queries on encrypted databases, or electronic voting — scenarios where an error of ± 1 would be unacceptable. The trade-off is that they can't represent real numbers directly: any computation with decimals requires manually managing fixed-point arithmetic.

CKKS accepts a small approximation error ($\sim 10^{-7}$) in exchange for natively encoding real numbers. That made it the reference scheme for *machine learning* and statistics on encrypted data, where that error is irrelevant compared to the noise in the data itself. It isn't suitable when results must be exact.

TFHE: Logic gates on the torus. TFHE completely abandons the SIMD model. Its mathematical foundation is different: instead of exploiting the roots of the polynomial x^N+1 to create independent slots (as the RLWE family does), it encrypts each bit as a position on the *torus* $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ — a circular structure where only the decimal part of the number matters — and evaluates the computation as a circuit of logic gates, one at a time

DRAFT

(Figure 5.14). Each gate runs an automatic bootstrapping, which eliminates the concept of a "level budget": the circuit can be arbitrarily deep without prior planning.

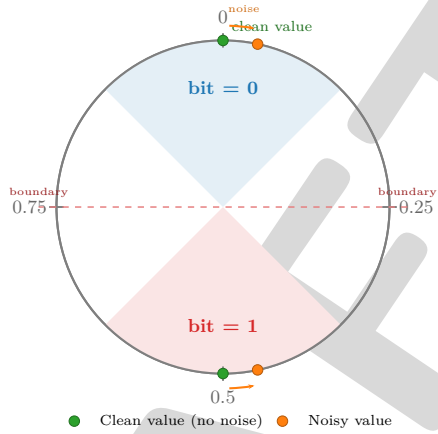


Figure 5.14: Encoding on the torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$. Bit 0 is encoded near 0.0 (top) and bit 1 near 0.5 (bottom). Noise displaces the position, but the bit is recovered as long as it doesn't cross the decision boundary.

TFHE's strength is exactly CKKS's weakness: comparisons, conditionals, and control flow. An `if (x > y)` that in CKKS requires approximating the sign function with a high-degree polynomial — consuming many levels —, in TFHE is evaluated directly bit by bit. The trade-off: without SIMD, adding two 32-bit integers requires chaining hundreds of logic gates where CKKS would execute a single operation. Modern libraries like Concrete (Zama) mitigate this by packing n-bit integers per ciphertext and using *programmable bootstrapping* — which applies any function during the noise-cleaning process, at no additional cost — to evaluate nonlinear functions like ReLU or comparisons in a single step.

Comparison. Table 5.3 summarizes the differences among the four schemes along the axes we've discussed:

DRAFT

Table 5.3: Main FHE schemes: foundations, capabilities, and trade-offs.

Scheme	Base	Native data type	Strength	Main limitation
BFV	RLWE	Exact integers	Exact integer arithmetic, SIMD	No reals; slow bootstrap
BGV	RLWE	Exact integers	Like BFV, more efficient at scale	No reals; slow bootstrap
CKKS	RLWE	Approx. reals	Floating point, SIMD	Accumulated error; inexact
TFHE	LWE + RLWE	Bits / small integers	Comparisons, conditionals; LUTs	No SIMD; slow arithmetic

In our example we used CKKS because we need to operate on real numbers (salaries) and can tolerate minimal approximation error — exactly the scenario it was designed for. But if Elena's next request were to identify the patient with the highest risk score, comparing encrypted values would stop being a minor detail, and Daniel would have to evaluate TFHE instead. If, instead, tireless Elena's requirement were to count encrypted votes with zero error margin, BFV or BGV would be the candidates.

Review Questions

5.24. Ask an LLM: *"Explain the difference between partially homomorphic, somewhat homomorphic, and fully homomorphic encryption. Give me a concrete example of each."* Is the answer accurate?

5.25. If `coeff_mod_bit_sizes=[60, 40, 40, 40, 60]`, how many chained multiplications does the scheme support?

5.26. BFV, BGV, and CKKS share the same RLWE base but differ in message encoding. Describe what each encodes, its main strength, and its most important limitation. How does TFHE differ from the three — both in foundations and in computation model?

DRAFT

DRAFT

Recommended Readings

Several weeks have passed since Elena walked into the room with her folder full of sticky notes. The contract with the vendor got signed, the risk analysis now runs every night over encrypted data, and legal — to everyone's surprise — signed off without asking for a single extra meeting. Daniel has gone from "messing around" with an unfamiliar technology to keeping it running in production, and along the way he's had to learn far more lattice algebra than he ever expected to need in his whole life. The Friday Elena called him in a panic because "the numbers were overflowing" made it clear that understanding the *what* isn't enough: you need the *why* to be able to debug and extend the system. And Elena has already floated her wish list to Daniel — comparing patients to prioritize the risk model, something CKKS doesn't handle well and that will push Daniel to evaluate TFHE, or to accept the one-second-per-operation cost of bootstrapping. For anyone who wants to go further than this chapter covers, here are the readings that make it possible.

The best starting point is Gentry (2010), where Gentry explains his own breakthrough in accessible language, without heavy formalism; it's ideal for understanding *what* FHE solves and *why* bootstrapping works. As a next step, Vaikuntanathan (2011) is an invited talk at FOCS that delves deeper into the mechanics of FHE without requiring advanced cryptography, explaining the evolution from partially homomorphic encryption to full FHE with exceptional clarity. For a comprehensive and up-to-date view of the field, Marcolla et al. (2022) offers an exhaustive review published in *Proceedings of the IEEE* covering all modern schemes (BGV, BFV, CKKS, TFHE), hardware acceleration, and real-world deployments.

Although FHE guarantees data confidentiality, it doesn't guarantee integrity — the second pillar of security (confidentiality, integrity, and availability) —: it doesn't guarantee that the server computed correctly, or that the result wasn't tampered with. The natural complement for adding

DRAFT

integrity is zero-knowledge proofs (Goldwasser, Micali, and Rackoff 1989; Thaler 2022).

However, FHE protects the data but not the program: the server knows what program it's running. To understand where this technology fits in the broader landscape of private computation, Barak (2016) offers an accessible introduction written about secure computation and obfuscation by one of the field's most prominent researchers. The paper places FHE alongside two complementary primitives. The first is *secure multiparty computation* (MPC), where several parties jointly compute without any of them seeing the others' data; unlike FHE, MPC is widely deployed in production today — with an overhead on the order of 10–100× over plaintext — and has real applications in fraud detection between banks or genomic data analysis between institutions. The second is *indistinguishability obfuscation* (iO), which would allow encrypting the program's logic as well — the scenario where neither the data nor the code is visible to the server —, but today it's completely impractical (several orders of magnitude slower than FHE itself), which is why it doesn't have its own chapter in this book.

DRAFT

References

Barak, Boaz. 2016. "Hopes, Fears, and Software Obfuscation." *Communications of the ACM* 59 (3): 88–96. https://doi.org/10.1145/2757276.

Gentry, Craig. 2009. "A Fully Homomorphic Encryption Scheme." PhD thesis, Stanford University. https://crypto.stanford.edu/craig/.

———. 2010. "Computing Arbitrary Functions of Encrypted Data." *Communications of the ACM* 53 (3): 97–105. https://doi.org/10.1145/1666420.1666444.

Goldwasser, Shafi, Silvio Micali, and Charles Rackoff. 1989. "The Knowledge Complexity of Interactive Proof Systems." *SIAM Journal on Computing* 18 (1): 186–208. https://doi.org/10.1137/0218012.

Marcolla, Chiara, Victor Sucasas, Marc Manzano, Riccardo Bassoli, Frank H. P. Fitzek, and Najwa Aaraj. 2022. "Survey on Fully Homomorphic Encryption, Theory, and Applications." *Proceedings of the IEEE* 110 (10): 1572–1609. https://doi.org/10.1109/JPROC.2022.3205665.

Rivest, Ronald L., Leonard Adleman, and Michael L. Dertouzos. 1978. "On Data Banks and Privacy Homomorphisms." *Foundations of Secure Computation*, 169–80.

Thaler, Justin. 2022. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security. Now Publishers. https://doi.org/10.1561/3300000030.

Vaikuntanathan, Vinod. 2011. "Computing Blindfolded: New Developments in Fully Homomorphic Encryption." In *Proceedings of the 52nd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 5–16. https://doi.org/10.1109/FOCS.2011.98.

DRAFT

DRAFT

Solutions

2.1. BFV operates on exact integers (decryption recovers the exact value via rounding) and is suitable for database queries or counting. CKKS operates on approximate real numbers (accepts a small error of $\sim 10^{-6}$) and is the reference scheme for machine learning and statistics. Verify that the LLM mentions both share the RLWE base and that the key difference is in message encoding.

2.2. `poly_modulus_degree=16384` doubles the security and allows more multiplicative levels, but each operation takes longer (polynomials are twice as large). With `4096`, operations are faster but you get fewer multiplicative levels and lower security. Execution time scales approximately as $O(n \log n)$ per NTT operation.

2.3. The formula $E[X^2] - (E[X])^2$ needs one element-wise multiplication (x_i^2) and one multiplication of the mean result by itself — two ciphertext $\times$ ciphertext multiplications. The alternative $\frac{1}{n}\sum(x_i - \bar{x})^2$ first requires computing $\bar{x}$ (sums and rotations), then subtracting it from each x_i (an addition by constant), and finally squaring each difference — it looks similar in multiplication count, but it first requires materializing the mean in every slot (extra rotations). In CKKS, the first formula is generally more efficient: fewer rotations and the same number of multiplicative levels.

2.4. **(a) Direct version — 2 links.** `mean_x = ct_x.sum() * (1/n)` consumes a link, just like `mean_y = ct_y.sum() * (1/n)` in the parallel branch; `mean_x * mean_y` consumes a second link, leaving that branch at level 2. On the other side, `ct_x * ct_y` consumes a link and `.sum() * (1/n)` consumes another, also leaving that branch at level 2: both reach the same link, with no mismatch between them — exactly like the naive variance — and with the same risk of silent overflow if the result doesn't fit the bit margin left at that last link. **(b) Optimized version — 1 link.** By also encrypting X/n and Y/n alongside X and Y (as we did with `ct_n`

DRAFT

for the variance), `mean_x = ct_xn.sum()` and `mean_y = ct_yn.sum()` consume no link at all — they're just rotations — so `mean_x * mean_y` consumes the only link in that branch; on the other side, `ct_x * ct_yn` already bakes the division by n into one operand and consumes the only link in its branch. Both branches end at level 1, with one extra 40-bit link of margin compared to the naive version. **(c) Zero links?** Not possible: unlike the public weights of Recipe 3, here X and Y are both private, so every cross term — $X \cdot Y$ or $E[X] \cdot E[Y]$ — is necessarily a ciphertext×ciphertext multiplication. Covariance is a bilinear function of two encrypted quantities, and that single link is the theoretical minimum that no circuit reordering can remove.

2.5. Because `features_cifrados * pesos` multiplies a ciphertext by plaintext constants, it's enough to multiply each component of the pair by the constant. No relinearization is needed, which is the most expensive part of a ciphertext × ciphertext multiplication, although one level is still consumed by the rescaling and the error grows in proportion to the constant. If the weights were also encrypted, it would be a ciphertext × ciphertext multiplication, which additionally requires relinearization and adds the other ciphertext's contribution to the error.

2.6. With 10,000 features and $N_{\text{slot}} = 4096$, you need $\lceil 10{,}000/4096 \rceil = 3$ ciphertexts to pack all the features. The prediction requires 3 multiplications by constants (one per ciphertext) and 3 rotation-based sums (one per ciphertext), and finally adding the 3 scalar results — a trivial operation on decrypted values, or an additional homomorphic addition.

2.7. It depends on how you write the polynomial. Evaluated term by term, $a \cdot x^3$ needs 3 levels: x^2 (level 1), $x^3 = x^2 \cdot x$ (level 2), and the multiplication by the constant a, which in CKKS is also encoded with scale Δ and consumes a level (level 3). Reordering it as $(a \cdot x) \cdot x^2$, where $a \cdot x$ and x^2 are computed in parallel at level 1, 2 levels suffice, since $b \cdot x^2$ and $c \cdot x$ get no further than level 2, and additions, including the constant d, consume no levels. The minimum configuration would be `coeff_mod_bit_sizes=[60, 40, 40, 60]` with `poly_modulus_degree=8192`; the direct version would require `[60, 40, 40, 40, 60]`.

2.8. The explicit error is a security decision: in a medical system, a silent "approximate" result could cause a misdiagnosis; in finance, a transaction with corrupted data could go unnoticed. The *fail-fast* principle ensures errors are caught in development, not production. A system that stops is preferable to one that produces plausible but incorrect results.

2.9. With depth 20: dimensioning 20 levels requires an enormous

DRAFT

`poly_modulus_degree` (32768) to maintain security, making every operation very slow. With 5 levels plus bootstrapping every 5 multiplications, you need 4 bootstraps (~4 s extra), but each individual multiplication is faster (smaller polynomials). For deep circuits, periodic bootstrapping is usually more efficient than static dimensioning, especially when the depth isn't known in advance.

2.10. Bootstrapping homomorphically evaluates the decryption circuit $(b - a \cdot s)$, which requires encrypting the secret key s under itself. *Circular security* is the assumption that publishing $\mathrm{Enc}_s(s)$ doesn't weaken the scheme. It isn't proven in general — it's an additional assumption that all FHE schemes rely on. Verify that the LLM distinguishes between this assumption and standard IND-CPA security.

2.11. TenSEAL is built on top of Microsoft SEAL, which for years deliberately focused on *leveled* encryption (no bootstrapping) to keep its API simple and its codebase easier to audit and maintain. Adding bootstrapping would have required taking on the additional circular-security assumption and managing much more complex level budgets, in exchange for a feature that most of TenSEAL's use cases (statistics, lightweight ML) don't need if levels are dimensioned properly from the start. OpenFHE, by contrast, started as a research project aiming to cover the full state of the art in FHE, so it accepts that extra complexity in exchange for supporting circuits of arbitrary depth.

2.12. The server must process *all* records because the query vector is encrypted — it can't distinguish the zeros from the one. If it processed only a subset, it would reveal by elimination that the queried position is in that subset, breaking the query's privacy.

2.13. With $N_{\text{slot}} = N/2 = 4096$ slots per ciphertext and 10,000 records, the client needs $\lceil 10{,}000/4096 \rceil = 3$ ciphertexts (one with the 1 in the correct position, the other two with only zeros). The server must process all 3 ciphertexts — it can't discard any without leaking information about the query.

2.14. Each additional block requires its own multiplication and sum over N_{slot} records, so the server's total work is the sum of the work across all blocks — still the same $O(n)$ as before, just spread across more operations on smaller ciphertexts. Private-retrieval systems that partition by public categories (hospital, year, etc.) narrow the pool of candidate records at the cost of revealing that category to the server, assuming it isn't sensitive information.

3.1. Current unsatisfactory alternatives: (1) In healthcare, data is

DRAFT

anonymized or NDAs are used, but anonymization is fragile (patients can be re-identified by cross-referencing variables) and agreements don't prevent the provider from seeing plaintext data. (2) In finance, controlled environments (*data clean rooms*) and legal agreements are used, but the recipient still accesses the actual data. (3) In multi-organization ML, data collection is centralized, exposing individual data. FHE would eliminate these exposures by always operating on encrypted data.

3.2. Companies and libraries: Microsoft SEAL (C++, BFV/CKKS), IBM HELib (BGV/CKKS), Zama (Concrete, TFHE), Google (FHE transpiler for C++), Duality Technologies. The maturity state in 2024 is *limited production*: there are real use cases (ML on encrypted data, financial computation) but the overhead is still high (10^4–$10^5\times$) for most applications.

4.1. The cost is acceptable when the value of privacy exceeds the computational cost: genomic data, proprietary financial models, electronic voting, computation on regulated medical data. It's not acceptable for low-latency applications (video games, high-frequency trading) or where data is not sensitive.

4.2. The main advances between 2015 and 2024 were: faster bootstrapping (TFHE achieves millisecond evaluations), CKKS for approximate arithmetic (2017), FHE compilers that transform normal code into homomorphic code (Google, Zama), and hardware acceleration (Intel HEXL, DARPA DPRIVE projects). The performance gap was reduced from $\sim 10^6\times$ to $\sim 10^4\times$ for many operations.

4.3. RLWE-based FHE schemes only offer two native operations: polynomial addition and multiplication. A comparison like `x > y` is not an arithmetic operation — it requires evaluating a nonlinear function (sign). To implement it, the sign function is approximated by a high-degree polynomial (typically degree 10-30), and each multiplication of that polynomial consumes a level from the budget. A simple `if (x > y)` can cost between 4 and 10 multiplicative levels.

5.1. SVP asks to find the shortest nonzero vector in the lattice; CVP asks to find the nearest lattice point to a given point in space. CVP is at least as hard as SVP because, if CVP could be solved efficiently, it could be used to solve SVP: simply query CVP with the origin and discard the zero vector. The intuition for why SVP may be easier is that its reference point (the origin) belongs to the lattice itself, which allows exploiting internal symmetries of the structure (translation invariance, for example). In CVP, the target point is arbitrary and "breaks" those symmetries — it's a problem parameterized by an external input unrelated to the periodic structure of

DRAFT

the lattice.

5.2. RSA relies on integer factorization, which Shor's algorithm solves in polynomial time on a quantum computer. Lattice problems (SVP, CVP) have no known efficient quantum analog: the best quantum algorithms remain exponential in the lattice dimension. That's why lattices are the foundation of post-quantum cryptography and, by extension, of FHE.

5.3. In the ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$, multiplying by a polynomial $a(x)$ is equivalent to multiplying by a negacyclic matrix determined by the coefficients of $a(x)$. This allows replacing the random LWE matrix A (N^2 integers, $O(N^2)$ multiplication) with a single polynomial (N integers, $O(N \log N)$ multiplication via NTT). With $N = 8192$, keys are reduced ~8,000× and operations are accelerated ~1,000×.

5.4. The RLWE equation is $b = a \cdot s + e$, where e is a small error. Without the error, recovering s would be trivial linear algebra (polynomial division). With it, b is a point *near* the lattice generated by the ring R_q, but not exactly on it — recovering s is equivalent to finding the nearest point, which is CVP. Formally, Lyubashevsky, Peikert, and Regev proved that solving RLWE in the average case allows solving approximate SVP on ideal lattices in the worst case. Verify that the LLM mentions the reduction applies to ideal lattices (those with the ring structure R_q, not general lattices) and that the noise e is what turns a trivial problem into an intractable one.

5.5. The unfolded product is $a(x) \cdot b(x) = 3x^5 + 8x^4 + 4x^3 + x^2 + 2x$. Applying $x^4 = -1$ (so $x^5 = -x$): $8x^4 \to -8$ and $3x^5 \to -3x$, leaving $4x^3 + x^2 - x - 8$. Reducing the coefficients modulo 5 ($-1 \equiv 4$, $-8 \equiv 2$) gives $4x^3 + x^2 + 4x + 2$.

5.6. The polynomial $x^N + 1$ has N complex roots, but since the message polynomial's coefficients are real, evaluations at conjugate roots are also conjugates: $m(\bar{\omega}) = \overline{m(\omega)}$. The roots group into $N/2$ conjugate pairs, and each pair contributes a single independent value. That's why there are $N/2$ slots, not N.

5.7. The remaining 4091 slots contain zeros. The cost of operations is unaffected: a homomorphic addition or multiplication processes *all* $N/2 = 4096$ slots in parallel, whether or not they're occupied. The cost is per ciphertext, not per datum — empty slots simply add or multiply zeros at no additional cost.

5.8. With $\Delta = 10$, the product $\Delta \cdot m(x)$ is $40 + 14.14\,x - 14.14\,x^3$, which rounds to $40 + 14\,x - 14\,x^3$. Its evaluations are $40 + 14\sqrt{2} \approx 59.8$ and $40 - 14\sqrt{2} \approx 20.2$, that is, 5.98 and 2.02 once the scale is undone, compared

DRAFT

with the original data 6 and 2. The rounding error goes from a few ten-thousandths (with $\Delta = 1000$) to a few hundredths, which illustrates that Δ determines how many decimals survive.

5.9. Yes, it's possible, but not in a single ciphertext: with $N = 4096$ ($N_{\text{slot}} = 2048$) or $N = 8192$ ($N_{\text{slot}} = 4096$), a single ciphertext never holds more than N_{slot} values. For a hundred thousand records you need to split them into $\lceil 100{,}000/N_{\text{slot}} \rceil$ blocks — 49 with $N = 4096$, 25 with $N = 8192$ — encrypt each block separately, apply the operation to each ciphertext, and combine the partial results, just as in the 10,000-feature example from Recipe 3. Who does that splitting depends on the library: OpenFHE rejects it with an error and leaves it to the programmer, while TenSEAL detects it and resolves it internally by creating several ciphertexts — though operations like matrix multiplication stop being available as soon as the vector spans more than one.

5.10. Without the random polynomial a, two ciphertexts $c_1 = s + e_1 + m_1$ and $c_2 = s + e_2 + m_2$ would allow the attacker to compute $c_1 - c_2 \approx m_1 - m_2$, revealing the difference between messages. The polynomial a, different in each encryption, acts as a mask that makes each ciphertext indistinguishable from random noise.

5.11. In homomorphic addition, errors add: $e_{\text{sum}} = e_1 + e_2$. If initial errors are on the order of ϵ, after k additions the error is $k\epsilon$. In multiplication, errors multiply with the messages: $e_{\text{mult}} \approx e_1 \cdot m_2 + e_2 \cdot m_1$, growing proportionally to the message sizes. This drastically limits the number of chained multiplications before noise overflows the modulus.

5.12. Homomorphic multiplication produces a triplet (d_0, d_1, d_2) that depends on s^2, instead of the standard pair (a, b) that depends only on s. Relinearization uses a public key that encrypts s^2 to convert the triplet into a standard pair. Without it, each multiplication would increase the ciphertext's degree in s, making it unmanageable after a few operations.

5.13. The logic `if (m < 0) then m = -m` computes the absolute value: $|m| = m \cdot \text{sign}(m)$. Since CKKS doesn't support conditional branches, the sign function must be approximated with a polynomial (for example, a degree 15–27 Chebyshev approximation). The implementation evaluates that polynomial on the ciphertext — $\text{Enc}(\text{sign}(m)) \approx p(\text{Enc}(m))$ — and then multiplies the result by $\text{Enc}(m)$: `enc_abs = enc_m * poly_sign(enc_m)`. Evaluating the polynomial consumes $\lceil \log_2(\text{degree}) \rceil \approx 4$–5 levels via the Paterson–Stockmeyer algorithm, plus one additional level for the final multiplication: between **5 and 10 multiplicative levels** in total, compared to a single instruction in conventional code.

DRAFT

5.14. The summation tree needs $\log_2(4096) = 12$ rotations and 12 homomorphic additions. At each step, the vector is rotated by a power of 2 $(1, 2, 4, \ldots, 2048)$ and the result is added to the original, so that after 12 steps each slot contains the sum of all elements.

5.15. The substitution $x \to x^k$ that implements the rotation transforms the ciphertext $(a(x), b(x))$ into $(a(x^k), b(x^k))$, which is encrypted under the key $s(x^k)$ instead of $s(x)$. To decrypt with the original key, a *key-switching* is needed that requires precomputed material — the Galois key — specific to each value of k. Different rotation amounts correspond to different exponents k, and each needs its own key.

5.16. In a CPU SIMD register, a circular shift reorders register positions with no cryptographic cost — it's a direct operation on bits. In FHE, rotation achieves the same logical effect (shifting vector positions), but requires an automorphism on the polynomial followed by key-switching, which adds noise and needs a precomputed Galois key. The similarity is functional: both move data within a "wide register." The difference is cost: nanoseconds on CPU, milliseconds in FHE, plus the need for additional keys.

5.17. In CKKS, each multiplication doubles the message scale from Δ to Δ^2 and amplifies the error by a factor of Δ. Rescaling divides the entire ciphertext by Δ, restoring the standard scale and eliminating the extra 40 bits of error. Without rescaling, the scale would grow exponentially and the modulus q would be exhausted after a single multiplication.

5.18. The circuit $a \cdot b + c \cdot d$ has two independent (parallel, not chained) multiplications followed by an addition. The multiplicative depth is 1. A single level suffices: `coeff_mod_bit_sizes=[60, 40, 60]` would be enough.

5.19. Because evaluating a polynomial at a point is a linear operation, and the slots are nothing more than the evaluations of the polynomial at the roots of $x^N + 1$. If all the coefficients are divided by Δ, each evaluation is divided by Δ independently, and no slot gets mixed with another.

5.20. The modulus of a freshly encrypted ciphertext has $60+30+30+30 = 150$ bits and, after each multiplication with its rescaling, drops to 120, 90, and 60 bits. At the end, $60 - 30 = 30$ bits remain for the integer part, about a billion. You gain one level (three multiplications instead of two) with only 10 more bits of total modulus (210 versus 200, still below the 218 bits that $N = 8192$ allows at 128-bit security, so there's no need to increase N), in exchange for about 9 decimal digits of precision instead of 12.

5.21. Bootstrapping evaluates the decryption circuit $(b - a \cdot s)$ homomor-

DRAFT

phically, which requires operating with s in the encrypted domain. This needs an encryption of s under the key s itself. This requires the *circular security* assumption: that encrypting the key under itself doesn't weaken the scheme. All current FHE schemes assume this property, although it hasn't been proven in general.

5.22. Dimensioning parameters to avoid bootstrapping requires a modulus q proportional to the number of multiplications, which demands an ever-larger `poly_modulus_degree` to maintain security. For very deep circuits, the polynomials would be so large that each operation would be slower than periodic bootstrapping. Bootstrapping is unavoidable when the circuit depth is not known in advance or is too large to dimension statically.

5.23. Homomorphic operations only see slots, but the modular reduction has to be applied to the coefficients. *CoeffsToSlots* places the coefficients into the slots by homomorphically evaluating the packing transform, using rotations and multiplications by constants. It consumes several additional levels, which are subtracted from the newly restored budget.

5.24. Partial: supports ONE type of operation without limit (RSA→multiplications, Paillier→additions). Somewhat (SHE): supports additions AND multiplications but with a limited number of operations before noise prevents decryption. Fully (FHE): supports additions and multiplications without limit, thanks to bootstrapping that periodically resets the noise. Verify that the LLM doesn't confuse SHE with FHE.

5.25. The intermediate levels (`40, 40, 40`) consume one multiplicative depth level each. With three intermediate 40-bit levels, the scheme supports **3 chained multiplications**. The extreme levels (`60`) are reserved for scale (encoding) and decryption.

5.26. *Encoding*: BFV scales the integer message by $\lfloor q/t \rfloor$ and recovers the exact result by rounding; BGV does the same with inverse modulus management; CKKS scales by Δ and accepts an approximation error to represent reals. *Strengths*: BFV/BGV guarantee exact integer arithmetic; CKKS enables floating point and is natural for ML. *Limitations*: BFV/BGV can't represent reals; CKKS accumulates error. TFHE differs on both axes: it uses LWE (not RLWE) — integer vectors instead of polynomials — and encodes a single bit per ciphertext on a torus. Its computation model is based on logic gates with automatic bootstrapping after each gate (~10 ms), making it ideal for comparisons and conditionals but inefficient for massive arithmetic (no SIMD parallelism).

DRAFT